

UNIVERSITÀ DEGLI STUDI DI ROMA “TOR VERGATA”

Macroarea di Ingegneria
Corso di Laurea Magistrale in Ingegneria Informatica

Prodotto matrice densa per multivettore in MPI e CUDA

Progetto di fine corso

Sistemi di Calcolo Parallelo e Applicazioni
A.A. 2025/2026

Autori

Alessandro Vassallo
0366572

Andrea Galluzzi
0365026

Codice sorgente: <https://github.com/AleVassa00/parallel-computing-project>

Versione della relazione del 16 settembre 2026

Bozza di lavoro. I marcatori in rosso segnalano i punti che dipendono da campagne di misura non ancora concluse. Per la versione di consegna, commentare `\todovisibiletrue` in `preambolo.tex`.

Indice

1	Introduzione	1
1.1	Obiettivo	1
1.2	Struttura della relazione	2
2	Il nucleo di calcolo	3
2.1	Che cosa calcola il nucleo	3
2.1.1	Conteggio delle operazioni	3
2.2	Come vengono memorizzati i dati	4
2.3	Ordine dei cicli e riuso dei dati	5
2.3.1	I due schemi candidati	6
2.3.2	Intensità rispetto al traffico di A	6
2.3.3	Il limite rispetto al traffico di A	7
2.3.4	Località degli accessi	7
2.4	Interfaccia comune dei backend locali	7
2.4.1	Ciclo di vita del backend	8
2.4.2	Contesto opaco del backend	8
2.4.3	Informazioni sperimentali	9
2.4.4	Selezione del backend	9
3	Parallelizzazione MPI	10
3.1	Che cosa significa parallelizzare in MPI	10
3.2	Perché si taglia A , e perché in due direzioni	11
3.2.1	Che cosa tagliare	11
3.2.2	Come tagliare: due possibilità monodimensionali	11
3.2.3	Il taglio bidimensionale	11
3.2.4	Perché i processi vengono organizzati a griglia	12
3.3	Decomposizione a griglia bidimensionale	12
3.3.1	I due sotto-comunicatori	13
3.3.2	Chi fa da root, e perché la scelta è verificata	13
3.4	Partizionamento degli indici	14
3.4.1	Le funzioni	14
3.5	Layout dei blocchi locali	14
3.5.1	Perché i conteggi delle collettive tornano	14
3.5.2	Chi contiene che cosa, e quando	15
3.5.3	Perché il multivettore non viene mai tagliato lungo k	15
3.6	Distribuzione iniziale dei dati	15
3.7	L'algoritmo distribuito	16
3.8	Modello di costo della comunicazione	17
4	Il nucleo di CPU	19

4.1	Tre backend, una sola idea	19
4.2	Il backend di riferimento	19
4.2.1	Perché specializzare k a compile-time	19
4.2.2	Il fallback generico, e perché la traccia lo richiede	20
4.3	Perché $KB = 32$	21
4.4	Ciò che il compilatore vede	21
4.5	Il primo limite: la rilettura di X	21
4.6	Secondo backend: X mattonellata in cache	22
4.7	Terzo backend: register blocking sulle righe	22
5	I backend CUDA	24
5.1	Struttura comune	25
5.2	Misurazione del backend CUDA	26
5.3	Preprocessing e occupazione della GPU	27
5.3.1	Gestione della memoria disponibile	27
5.4	<code>cuda_naive</code>	27
5.5	<code>cuda_warp</code> : un warp per riga	29
5.5.1	Gestione di k	30
5.5.2	La maschera dello shuffle	30
5.5.3	Il problema residuo: gli accessi a X	30
5.6	Primo rimedio: cambio di layout di X	31
5.7	<code>cuda_warp_smem</code> : staging di X in shared memory	31
5.7.1	Caricamento cooperativo di X	32
5.7.2	Riuso del tile fra i warp del blocco	32
5.7.3	Bank conflict e padding della shared memory	33
5.7.4	Scelta della dimensione del tile	34
5.7.5	Sincronizzazione e warp inattivi	35
5.8	Due rimedi allo stesso problema	35
5.9	<code>cublas</code> : il riferimento esterno	36
5.9.1	Adattamento fra row-major e column-major	36
5.9.2	Significato del confronto	37
5.10	Più rank MPI sulla stessa GPU	37

Capitolo 1

Introduzione

1.1 Obiettivo

Il progetto realizza un nucleo di calcolo parallelo per il prodotto tra una matrice densa e un multivettore:

$$Y \leftarrow AX, \quad A \in \mathbb{R}^{M \times N}, \quad X \in \mathbb{R}^{N \times k}, \quad Y \in \mathbb{R}^{M \times k}. \quad (1.1)$$

Il multivettore X può essere visto come una matrice densa con un numero k di colonne molto più piccolo rispetto alle dimensioni della matrice A . Il codice supporta valori generici di k ed è stato collaudato sui cinque valori richiesti dalla traccia,

$$k \in \{3, 6, 8, 20, 32\}.$$

Le dimensioni globali M e N non sono fissate in fase di compilazione, ma vengono scelte a runtime. Il nucleo deve quindi adattarsi sia alla forma della matrice sia alla dimensione del multivettore.

La parallelizzazione è organizzata su due livelli. Il livello esterno è basato su MPI e distribuisce il problema fra più processi. La matrice A viene suddivisa su una griglia bidimensionale di processi, e le comunicazioni MPI rendono disponibili i dati necessari al calcolo dei contributi locali e alla successiva costruzione del risultato globale.

All'interno di ciascun processo, il prodotto sul blocco locale è demandato a un backend di calcolo attraverso un'interfaccia comune. Il progetto comprende implementazioni direttamente su CPU e diversi backend CUDA, questi ultimi utilizzati per sfruttare il parallelismo della GPU NVIDIA.

I due livelli operano quindi in modo gerarchico: MPI determina quali dati e quale parte del prodotto competono a ciascun processo, mentre il backend locale determina come quel prodotto viene eseguito all'interno del singolo processo.

Oltre alla correttezza numerica, il progetto ha come obiettivo il confronto prestazionale delle diverse strategie implementate. Le successive ottimizzazioni vengono perciò introdotte e misurate separatamente, così da poter collegare le differenze di prestazione alle specifiche scelte di distribuzione, accesso alla memoria e parallelizzazione.

Il lavoro non si limita alla realizzazione funzionale del prodotto. Le diverse implementazioni vengono confrontate sia dal punto di vista della correttezza numerica sia da quello delle

prestazioni. La correttezza viene verificata rispetto a un riferimento seriale, mentre le prestazioni vengono studiate al variare delle dimensioni della matrice, del numero di colonne del multivettore e della configurazione parallela.

La separazione tra livello MPI e backend locale permette di mantenere invariata la decomposizione distribuita del problema e di sostituire indipendentemente il nucleo di calcolo eseguito da ciascun processo. In questo modo le diverse implementazioni locali possono essere confrontate all'interno della stessa pipeline MPI.

La presenza di un k piccolo rispetto a M e N distingue inoltre il problema da una GEMM generale: ciascun elemento di A viene utilizzato per contribuire a più colonne del multivettore, rendendo il riutilizzo dei dati e il modo in cui questi vengono disposti in memoria aspetti centrali dell'ottimizzazione.

1.2 Struttura della relazione

La relazione introduce dapprima la decomposizione del prodotto e l'organizzazione dei dati, quindi descrive il livello distribuito MPI. I capitoli successivi analizzano separatamente i backend locali su CPU e GPU, mettendo in relazione le scelte implementative con il comportamento della gerarchia di memoria e con il parallelismo disponibile. Infine vengono presentate la metodologia sperimentale e l'analisi delle prestazioni ottenute.

Capitolo 2

Il nucleo di calcolo

Questo capitolo fissa tre cose che il resto della relazione dà per acquisite: che cosa il nucleo calcola e come se ne contano le operazioni, in che ordine conviene scandire i tre cicli, e da dove vengono i dati. Sono scelte fatte una volta sola, prima di qualunque backend: i capitoli su MPI, CPU e CUDA le ereditano senza rimetterle in discussione.

2.1 Che cosa calcola il nucleo

Il nucleo realizza il prodotto fra una matrice densa A e un multivettore X :

$$Y \leftarrow AX, \quad A \in \mathbb{R}^{M \times N}, \quad X \in \mathbb{R}^{N \times k}, \quad Y \in \mathbb{R}^{M \times k}. \quad (2.1)$$

Elemento per elemento, il risultato è definito da:

$$Y[i][c] = \sum_{j=0}^{N-1} A[i][j] X[j][c], \quad 0 \leq i < M, \quad 0 \leq c < k. \quad (2.2)$$

Le dimensioni M e N vengono scelte a runtime. Anche k è un parametro del problema e il codice deve supportarne valori generici. Come richiesto dalla traccia, il collaudo comprende nel particolare:

$$k \in \{3, 6, 8, 20, 32\}.$$

La caratteristica distintiva del problema è che k è piccolo rispetto alle dimensioni della matrice A . Il prodotto può quindi essere visto come l'applicazione della stessa matrice a un numero limitato di vettori, raccolti nelle colonne di X .

2.1.1 Conteggio delle operazioni

Per ogni elemento $Y[i][c]$ devono essere combinati N termini del tipo:

$$A[i][j] X[j][c].$$

Adottando la convenzione utilizzata dalla traccia, ogni termine viene conteggiato come due operazioni floating point, una moltiplicazione e un'addizione. Il lavoro associato a un elemento di Y è quindi pari a $2N$ FLOP.

Poiché Y contiene Mk elementi, il numero complessivo di operazioni associato al prodotto globale è:

$$\text{FLOP}(M, N, k) = 2MNk. \quad (2.3)$$

Se T indica il tempo medio necessario per una singola invocazione del nucleo, la prestazione viene espressa come:

$$\text{GFLOPS} = \frac{2MNk}{T \cdot 10^9}. \quad (2.4)$$

Le dimensioni M e N presenti nel numeratore sono sempre quelle della matrice globale, anche quando il prodotto viene distribuito fra più processi MPI: quindi il numero di operazioni associato al problema rimane invariato al variare della configurazione parallela.

La traccia richiede che T sia ottenuto ripetendo il nucleo più volte sulla stessa matrice e considerando il tempo medio per invocazione. Preprocessing, operazioni di I/O e trasferimenti dei dati da e verso la GPU non contribuiscono alla misura prestazionale ufficiale, anche se il progetto li misura separatamente per caratterizzarne il costo.

Nel caso distribuito, la definizione precisa delle fasi comprese nel tempo del prodotto e il modo in cui i tempi dei diversi rank vengono combinati sono descritti in ??.

2.2 Come vengono memorizzati i dati

Dopo la distribuzione, ciascun processo opera su tre buffer locali:

$$A_{\text{loc}} \in \mathbb{R}^{m_{\text{loc}} \times n_{\text{loc}}}, \quad X_{\text{loc}} \in \mathbb{R}^{n_{\text{loc}} \times k}, \quad Y_{\text{loc}} \in \mathbb{R}^{m_{\text{loc}} \times k}.$$

La rappresentazione di riferimento utilizzata dal progetto è **row-major**: gli elementi appartenenti alla stessa riga sono quindi memorizzati in posizioni consecutive.

L'interfaccia dei backend associa a ciascuna matrice una *leading dimension*, indicata per ciascuna matrice rispettivamente con `lda`, `ldx` e `ldy`. La leading dimension esprime, in numero di elementi, la distanza fra l'inizio di due righe consecutive del buffer. Gli elementi vengono quindi indirizzati come

$$A[i][j] = A_{\text{loc}}[i \cdot \text{lda} + j],$$

$$X[j][c] = X_{\text{loc}}[j \cdot \text{ldx} + c], \quad Y[i][c] = Y_{\text{loc}}[i \cdot \text{ldy} + c].$$

Una leading dimension uguale al numero di colonne corrisponde a una *rappresentazione compatta*. Se invece è maggiore, fra la parte utile di due righe consecutive rimangono alcuni elementi inutilizzati, che costituiscono il *padding*.

In particolare per la matrice locale A il progetto permette:

$$\text{lda} \geq n_{\text{loc}}.$$

Il numero n_{loc} specifica quindi quanti elementi della riga appartengono effettivamente alla matrice locale, mentre `lda` ne descrive la disposizione fisica in memoria. Separare le due

quantità permette di introdurre padding senza modificare né le dimensioni logiche del problema né l'interfaccia del backend.

La distinzione è utile anche durante la distribuzione MPI: un blocco bidimensionale estratto dalla matrice globale non è, in generale, una singola regione contigua del buffer row-major: i segmenti appartenenti a righe differenti sono separati dallo stride della matrice globale. La distribuzione utilizza datatype MPI derivati per descrivere tale struttura e depositare i dati nella rappresentazione locale rispettando la *lda* scelta. I dettagli della distribuzione sono descritti nel capitolo 3.

Per quanto riguarda invece X_{loc} e Y_{loc} l'interfaccia mantiene ugualmente esplicite le rispettive leading dimension, ma la pipeline distribuita utilizza buffer compatti, cioè con:

$$\text{ldx} = \text{ldy} = k.$$

Questa scelta semplifica le comunicazioni eseguite a ogni invocazione del prodotto. Le porzioni di X coinvolte nel broadcast e i contributi di Y coinvolti nella riduzione possono essere trattati come regioni contigue di memoria, senza introdurre padding o ulteriori descrizioni del layout nel percorso ripetuto dell'algoritmo.

La matrice A , al contrario, viene distribuita durante il preprocessing e rimane poi residente nel processo. Il suo buffer locale può quindi utilizzare una leading dimension più generale senza modificare le comunicazioni ripetute del prodotto.

La compattezza delle righe di X è inoltre rilevante per l'ordine dei cicli discusso nella sezione successiva, perché gli elementi:

$$X[j][0], X[j][1], \dots, X[j][k-1]$$

occupano posizioni consecutive in memoria.

Un backend CUDA considera anche una rappresentazione alternativa di X per colonne. Si tratta di una variante specifica, introdotta e discussa nel capitolo 5. Salvo quando indicato esplicitamente, nel seguito si assume il normale layout row-major.

2.3 Ordine dei cicli e riutilizzo dei dati

La eq. (2.2) coinvolge tre indici, i , j e c . Cambiare l'ordine con cui vengono percorsi non modifica il risultato matematico, ma modifica il modo in cui i dati vengono riutilizzati e acceduti in memoria. Due implementazioni equivalenti dal punto di vista numerico possono quindi avere comportamenti prestazionali molto diversi.

Schema A ($i \rightarrow j \rightarrow c$)

```
for i:
    acc[0..k) = 0
    for j:
        a = A[i][j]
        for c:
            acc[c] += a * X[j][c]
Y[i][0..k) = acc[0..k)
```

Schema B ($i \rightarrow c \rightarrow j$)

```
for i:
    for c:
        s = 0
        for j:
            s += A[i][j] * X[j][c]
Y[i][c] = s
```

2.3.1 I due schemi candidati

Lo *schema B* segue direttamente la definizione elemento per elemento. Per ogni riga i e per ogni colonna c di Y , percorre l'intera dimensione j e calcola:

$$Y[i][c] = \sum_j A[i][j]X[j][c].$$

Può quindi essere interpretato come l'esecuzione di k prodotti matrice-vettore indipendenti, uno per ciascuna colonna di X .

Lo *schema A* scambia invece i due cicli interni. Fissati i e j , l'elemento

$$a = A[i][j]$$

viene utilizzato per aggiornare tutti i k elementi della riga corrente di Y :

$$\text{acc}[c] += a X[j][c], \quad 0 \leq c < k.$$

In questo modo ogni utilizzo di $A[i][j]$ necessario alle k colonne del risultato viene concentrato nella stessa iterazione del ciclo su j .

Dal punto di vista del codice, lo schema B richiede ogni elemento di A una volta per ciascuna delle k colonne, mentre nello schema A ogni elemento viene caricato una volta e riutilizzato immediatamente per tutti i k accumulatori.

Questa distinzione riguarda il *numero di accessi* espresso dall'algoritmo. Il traffico effettivamente generato verso i diversi livelli della gerarchia di memoria può essere anche inferiore, perché le riletture dello schema B possono essere servite dalle cache. Lo schema A rende tuttavia il riuso esplicito nel codice, senza dipendere dalla capacità della cache di conservare i dati fra una colonna e la successiva.

L'efficienza con cui questo riuso viene realizzato dipende poi dal backend. In particolare, mantenere gli accumulatori in registri ed evitare spill in memoria diventa un aspetto importante delle implementazioni CPU e CUDA, discusso nei rispettivi capitoli.

2.3.2 Intensità rispetto al traffico di A

Per quantificare il vantaggio dello schema A è utile considerare separatamente il traffico associato alla matrice A .

Supponiamo, come modello ideale, che ogni accesso ad A che non viene riutilizzato esplicitamente debba essere soddisfatto dal livello di memoria considerato. Indicando con s la dimensione in byte di uno scalare, lo schema A richiede $MN s$ byte associati ad A , mentre lo schema B ne richiede $kMN s$.

Rapportando i $2MNk$ FLOP del prodotto a questo solo contributo si ottiene:

$$\mathcal{I}_A^{(A)} = \frac{2MNk}{MN s} = \frac{2k}{s}, \quad \mathcal{I}_A^{(B)} = \frac{2MNk}{kMN s} = \frac{2}{s}. \quad (2.5)$$

Queste quantità *non* rappresentano l'intensità aritmetica completa del kernel, perché non includono il traffico associato a X e Y e non modellano gli effetti delle cache. Servono invece a isolare il beneficio del riuso esplicito della matrice A introdotto dallo schema A.

Il fattore k mostra quanto l'ordine dei cicli possa aumentare il riuso esplicito della matrice dominante.

Per stabilire se questo sia sufficiente a spostare il kernel da un regime limitato dalla memoria a uno limitato dal calcolo è però necessario includere anche gli altri flussi di memoria e confrontare l'intensità complessiva con le caratteristiche dell'hardware. Questa analisi è svolta nel ??.

2.3.3 Il limite rispetto al traffico di A

Nel modello appena descritto, lo schema A raggiunge il minimo traffico possibile per la matrice A : ciascuno degli MN elementi deve essere reso disponibile almeno una volta per poter contribuire al prodotto.

Ulteriori ottimizzazioni non possono quindi ridurre questo limite algoritmico sulla matrice A , ma possono agire sul modo in cui la matrice X viene acceduta, sulla permanenza degli accumulatori nei registri e sull'effettivo sfruttamento della gerarchia di memoria.

2.3.4 Località degli accessi

Lo schema A presenta inoltre una proprietà favorevole nel codice sequenziale. Al crescere di j , gli elementi

$$A[i][0], A[i][1], \dots$$

sono attraversati nell'ordine con cui compaiono nella riga di A .

Per ogni j , il ciclo interno su c attraversa invece

$$X[j][0], X[j][1], \dots, X[j][k-1],$$

anch'essi consecutivi nel layout row-major adottato dal progetto.

Lo schema espone quindi una buona località spaziale per entrambi i buffer nel percorso sequenziale. Il modo in cui tale proprietà si traduce sull'hardware parallelo dipende però dalla mappatura del lavoro: l'ordine dei cicli descrive quali dati vengono visitati consecutivamente nell'esecuzione sequenziale, ma non dice ancora quali dati vengono elaborati nello stesso momento dai diversi thread. In particolare, sulla CPU conta quali elementi vengono elaborati consecutivamente da un core e quali cicli possono essere vettorizzati, mentre sulla GPU conta quali indirizzi vengono richiesti simultaneamente dalle lane di uno stesso warp.

La *coalescenza degli accessi* non può quindi essere dedotta dal solo ordine dei cicli e verrà analizzata separatamente per ciascun kernel CUDA.

2.4 Interfaccia comune dei backend locali

Dopo aver definito il prodotto locale, la rappresentazione dei dati e l'organizzazione di base del calcolo, resta da stabilire come il livello distribuito interagisca con le diverse implementazioni del nucleo locale.

Tutti i backend espongono la stessa interfaccia, definita in `src/kernel/kernel.h`. Il livello MPI può quindi utilizzare il nucleo locale senza conoscere i dettagli della sua implementazione, che può essere basata sulla CPU, su un kernel CUDA oppure su una libreria esterna come cuBLAS.

2.4.1 Ciclo di vita del backend

Le tre operazioni principali dell'interfaccia sono:

```
local_gemm_t *local_gemm_create(int m, int n, int k,
                                const scalar_t *A_loc,
                                int lda, int ldx, int ldy);

void local_gemm(local_gemm_t *ctx,
                const scalar_t *RESTRICT X, int ldx,
                scalar_t *RESTRICT Y, int ldy);

void local_gemm_destroy(local_gemm_t *ctx);
```

Listato 2.1: Operazioni principali dell'interfaccia definita in `src/kernel/kernel.h`.

Le tre funzioni corrispondono alle fasi di inizializzazione, esecuzione e finalizzazione del backend locale. La separazione non è soltanto organizzativa, ma riflette il diverso ciclo di vita dei dati coinvolti nel prodotto.

Il blocco locale A_{loc} rimane invariato durante tutte le ripetizioni del nucleo. Può quindi essere associato al contesto una sola volta durante `local_gemm_create`. I buffer relativi a X e Y , invece, appartengono alla singola invocazione e vengono forniti a `local_gemm` ogni volta che il prodotto deve essere eseguito.

La funzione `local_gemm_create` inizializza quindi il contesto del backend e prepara tutte le risorse che possono essere riutilizzate fra più invocazioni. Nei backend CPU ciò comprende principalmente le informazioni necessarie a descrivere il problema e il blocco locale di A . Nei backend CUDA vengono inoltre inizializzate le risorse del device, allocati i buffer persistenti e trasferita A_{loc} nella memoria della GPU.

La funzione `local_gemm` rappresenta invece il percorso ripetuto del calcolo. Riceve il blocco di X reso disponibile al processo dal livello distribuito e produce il corrispondente contributo locale in Y . Nei backend CUDA vengono riutilizzati i buffer predisposti durante l'inizializzazione: X viene trasferita sul device, viene eseguito il prodotto locale e il risultato viene infine riportato nella memoria host.

Al termine dell'esecuzione, `local_gemm_destroy` rilascia tutte le risorse associate al contesto, comprese le eventuali allocazioni e risorse CUDA.

Questa organizzazione evita di ripetere a ogni invocazione le operazioni associate ai dati persistenti e mantiene separati i costi di preparazione da quelli del percorso effettivamente ripetuto durante il benchmark.

2.4.2 Contesto opaco del backend

Lo stato specifico di ciascuna implementazione è racchiuso nel tipo opaco `local_gemm_t`. Il codice che utilizza l'interfaccia conosce soltanto un puntatore a tale tipo, mentre la struttura concreta viene definita separatamente da ciascun backend.

Questa scelta permette alle diverse implementazioni di mantenere informazioni e risorse differenti senza modificare il codice che le utilizza. Un backend CPU può, per esempio, conservare le dimensioni del problema e il riferimento al blocco locale di A , mentre un backend CUDA deve mantenere anche i puntatori ai buffer sul device e le risorse necessarie all'esecuzione e alla misurazione.

Il contesto costituisce quindi il confine fra l'interfaccia comune e lo stato specifico dell'implementazione. Il livello distribuito non deve conoscere come viene realizzato il prodotto locale: fornisce i dati attraverso `local_gemm` e riceve il risultato tramite la stessa interfaccia per tutti i backend.

2.4.3 Informazioni sperimentali

Oltre alle tre operazioni principali, l'interfaccia espone alcune funzioni di introspezione utilizzate dal driver di benchmark. Queste permettono di recuperare informazioni specifiche del backend, come tempi di inizializzazione, tempi di trasferimento, tempo del calcolo locale e parametri della configurazione effettivamente utilizzata.

Tali funzioni non partecipano alla definizione matematica del prodotto e non modificano il contratto di `local_gemm`. Servono esclusivamente a rendere osservabili separatamente i costi che compongono l'esecuzione. Il significato delle singole metriche e il modo in cui vengono aggregate sono descritti nel capitolo dedicato alla metodologia sperimentale.

2.4.4 Selezione del backend

L'implementazione del nucleo locale viene selezionata a tempo di compilazione. Tutti i backend rispettano la stessa interfaccia, per cui il driver e il livello MPI non devono essere modificati quando cambia l'implementazione utilizzata per il prodotto locale.

La configurazione di build controlla anche alcune proprietà comuni, come la precisione numerica, e parametri specifici dei singoli backend. Le configurazioni utilizzate nelle campagne di misura e la loro identificazione nei risultati vengono descritte insieme alla metodologia sperimentale.

L'interfaccia realizza quindi la separazione fra i due livelli introdotti nel capitolo 1: MPI stabilisce quali dati e quale contributo al prodotto appartengono a ciascun processo, mentre il backend locale stabilisce come quel contributo viene calcolato.

Capitolo 3

Parallelizzazione MPI

Fino a questo punto il prodotto è stato descritto come se avvenisse dentro un unico spazio di memoria: i cicli di sezione 2.3 leggono A e X e scrivono Y come se tutte e tre fossero lì, a disposizione. Questo capitolo rimuove quell'ipotesi. Il problema diventa far collaborare più processi che non condividono niente, e le scelte che ne derivano condizionano tutto il resto della relazione: la forma dei blocchi, che cosa viene cronometrato, e perfino quali configurazioni ha senso misurare.

3.1 Che cosa significa parallelizzare in MPI

MPI (*Message Passing Interface*) è uno standard per programmare macchine a **memoria distribuita**. Il modello di esecuzione è il seguente: lo stesso programma viene avviato in P copie, ciascuna un processo del sistema operativo a sé stante, con il proprio spazio di indirizzi **privato**. Nessun processo può leggere o scrivere una variabile di un altro. L'unico modo perché un dato passi da un processo all'altro è che qualcuno lo invii esplicitamente, con una chiamata di libreria, e qualcun altro lo riceva.

È una differenza sostanziale rispetto a un modello a memoria condivisa, dove i thread vedono gli stessi indirizzi e la parallelizzazione consiste essenzialmente nel ripartire le iterazioni di un ciclo. Qui ripartire le iterazioni non basta, e non è nemmeno la prima cosa da decidere: prima bisogna stabilire **chi possiede quale dato**. Da questa scelta discende tutto il resto, perché un processo può calcolare solo ciò per cui possiede gli ingredienti, e ogni ingrediente mancante è un messaggio da far viaggiare.

La parallelizzazione del nucleo si articola quindi in tre domande, in quest'ordine:

1. **decomposizione** - come si tagliano A , X e Y fra i processi;
 2. **località** - quale parte del risultato ciascun processo riesce a calcolare con i soli dati che possiede;
 3. **comunicazione** - che cosa manca, e in quale direzione deve viaggiare perché il risultato locale diventi il risultato giusto.
- Il **rank** è l'identificatore intero di un processo all'interno di un gruppo: è ciò che permette a un programma unico di comportarsi diversamente su processi diversi.
 - Un **comunicatore** è un gruppo di processi assieme a un contesto di comunicazione isolato. Ogni operazione MPI avviene *dentro* un comunicatore, e i rank hanno senso solo rispetto a quello: `MPI_COMM_WORLD` è il comunicatore predefinito che contiene tutti i processi, ma se ne possono costruire altri più piccoli, ed è esattamente ciò che il progetto fa.

- Un'operazione **collettiva** è un'operazione che tutti i processi di un comunicatore devono chiamare. Il progetto ne usa due: il *broadcast*, in cui un processo possiede un dato e al termine lo possiedono tutti; e la *reduce*, in cui ogni processo possiede un array della stessa forma, gli array vengono combinati elemento per elemento con un operatore associativo — qui la somma — e il risultato finisce su un solo processo.
- Il **root** è quel processo designato: la sorgente nel broadcast, il destinatario nella reduce.

3.2 Perché si taglia A , e perché in due direzioni

3.2.1 Che cosa tagliare

La prima decisione riguarda quale delle tre matrici vada distribuita. La risposta viene da ???: A ha MN elementi, X e Y ne hanno rispettivamente Nk e Mk , e con k piccolo per definizione A è più grande delle altre due di un fattore che cresce con la taglia del problema.

Replicare A su ogni processo sarebbe quindi la scelta peggiore possibile, e per due ragioni indipendenti: ogni nodo dovrebbe contenerla per intero, mettendo un tetto alla taglia massima trattabile a prescindere da quanti processi si usino; e ogni processo la rileggerebbe tutta, quando è proprio il traffico su A a determinare il tempo. A va dunque **tagliata**. X e Y , che sono piccole, possono invece permettersi di essere replicate lungo un asse, e vedremo che è esattamente ciò che conviene fare.

3.2.2 Come tagliare: due possibilità monodimensionali

Tagliare A significa assegnare a ogni processo una sottomatrice. Le decomposizioni più semplici la tagliano lungo una sola dimensione, e la eq. (2.2) dice subito che cosa comportano.

Taglio a bande di righe. Ogni processo possiede righe *interi* di A . Per calcolare $Y[i][c]$ serve tutta la riga i di A e tutte le righe di X : un processo che possiede righe intere possiede quindi tutti i fattori del primo tipo, ma gli serve X per intero. Il vantaggio è che il risultato che calcola è già *definitivo*: nessuna somma finale fra processi. Il costo è che X deve essere replicata su tutti i P processi.

Taglio a bande di colonne. Ogni processo possiede colonne di A , cioè, per ciascuna riga, solo una parte degli addendi della sommatoria. Gli basta quindi la porzione di X corrispondente alle proprie colonne — X viene tagliata e non replicata — ma ciò che produce è un risultato **parziale**, che riguarda tutte le righe di Y e va sommato con i parziali di tutti gli altri. La somma finale coinvolge P processi su un oggetto della taglia piena di Y .

Le due alternative sono speculari: la prima paga la replicazione di X su tutti, la seconda la riduzione di Y su tutti. In entrambe la comunicazione coinvolge sempre l'intero insieme dei processi, e questo è il difetto strutturale: aumentare P non rende i messaggi più piccoli, li rende soltanto più numerosi.

3.2.3 Il taglio bidimensionale

La terza possibilità taglia A in entrambe le direzioni: ogni processo possiede un *rettangolo* della matrice. Le due necessità di prima compaiono ora entrambe — servono le righe di X corrispondenti alle proprie colonne di A , e il proprio contributo a Y è parziale e va sommato — ma ciascuna coinvolge soltanto un **sottoinsieme** dei processi: quelli che condividono le stesse colonne di A per la prima, quelli che condividono le stesse righe per la seconda. Nessuna delle due comunicazioni è più globale.

È questa la decomposizione richiesta dalla traccia, ed è quella che il progetto implementa. Che sia effettivamente superiore alle due monodimensionali non è però qualcosa da accettare sulla fiducia: è una proprietà quantificabile, e la sezione 3.8 la dimostra con un modello di costo, mostrando che le due decomposizioni 1D ne sono i casi degeneri. Il confronto viene poi misurato direttamente in ??.

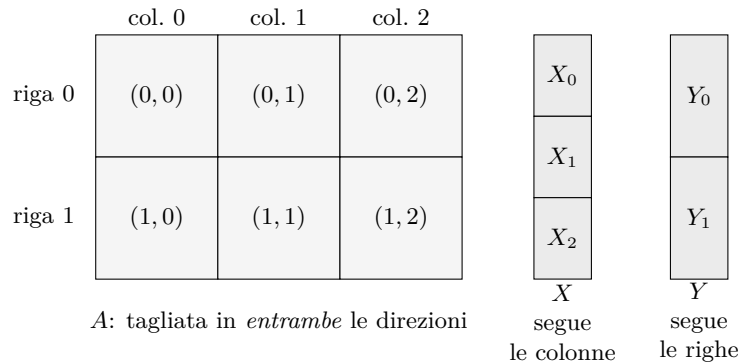


Figura 3.1: Decomposizione dei dati su una griglia 2×3 . Ogni processo possiede un *rettangolo* di A , individuato dalle sue coordinate nella griglia. Il multivettore X è tagliato per bande di righe, una per ciascuna banda di colonne di A ; Y per bande di righe, una per ciascuna banda di righe di A . Nessuna delle due è mai tagliata lungo k .

3.2.4 Perché i processi vengono organizzati a griglia

Resta un ultimo passaggio, che è di comodità ma non solo. Se il taglio del dato è bidimensionale, conviene che lo sia anche il modo in cui si identificano i processi: un processo individuato da una coppia di coordinate (r, c) invece che da un intero rende immediate le due relazioni che servono di continuo. «I processi che possiedono le stesse colonne di A » diventa «i processi della stessa colonna della griglia», e non un calcolo aritmetico sui rank da rifare ogni volta.

MPI offre questa struttura nativamente, sotto il nome di **topologia virtuale cartesiana**. È virtuale nel senso che non descrive come i nodi sono fisicamente cablati: descrive come il programma vuole *pensare* all'insieme dei propri processi, lasciando però all'implementazione la possibilità di sfruttare quell'informazione per disporli meglio sulla macchina reale.

3.3 Decomposizione a griglia bidimensionale

I P processi vengono organizzati in una griglia cartesiana $P_r \times P_c = P$, costruita con `MPI_Cart_create`. La chiamata restituisce un nuovo comunicatore nel quale ogni processo ha, oltre al proprio rank, una coppia di coordinate recuperabile con `MPI_Cart_coords`: sono le (r, c) della sezione precedente, ed è su di esse che il codice ragiona da qui in avanti.

Due parametri della chiamata meritano una nota:

- la griglia è dichiarata **non periodica**. La periodicità serve agli schemi in cui ogni processo comunica con i propri vicini e il bordo deve richiudersi sul lato opposto; qui le comunicazioni sono collettive lungo un'intera riga o colonna, e un collegamento fra l'ultimo processo e il primo non avrebbe alcun significato;
- `reorder = 1` autorizza l'implementazione a **rinumerare** i processi rispetto a `MPI_COMM_WORLD`, per far corrispondere meglio la griglia logica alla topologia fisica della macchina. È la libertà che rende utile una topologia virtuale, ma ha una conseguenza da non dimenticare: da quel momento l'unico rank valido è quello del nuovo comunicatore, e assumere che coincida con quello di partenza sarebbe un errore.

La forma della griglia può essere imposta dall'utente con i flag `-pr` e `-pc`. In assenza di indicazione, `grid_default_shape` sceglie la fattorizzazione più vicina alla forma quadrata, prendendo come P_r il più grande divisore di P non superiore a $\sqrt{P}$; ne segue $P_r \leq P_c$, e nel caso limite di P primo la griglia degenera in $1 \times P$. La ragione per cui il quadrato è un buon default, e i casi in cui non lo è, sono in sezione 3.8.

3.3.1 I due sotto-comunicatori

La griglia da sola non basta: le collettive vanno eseguite lungo una riga o lungo una colonna, non sull'insieme di tutti i processi, e una collettiva agisce su un comunicatore intero. Servono quindi comunicatori più piccoli, uno per ogni riga e uno per ogni colonna della griglia.

Li produce `MPI_Cart_sub`, a cui si indica quali dimensioni lasciar variare e quali tenere fisse. Fissando la coordinata di riga e lasciando variare quella di colonna si ottiene `row_comm`, che raccoglie i processi di una stessa *riga*; fissando la colonna si ottiene `col_comm`, che raccoglie quelli di una stessa *colonna*. Ogni processo appartiene esattamente a un `row_comm` e a un `col_comm`, e sono i due assi lungo cui viaggiano le uniche due collettive del cammino cronometrato.

3.3.2 Chi fa da root, e perché la scelta è verificata

Entrambe le collettive hanno bisogno di un root, e in entrambi i casi il processo giusto è determinato dal problema, non arbitrario: il broadcast di X deve partire dal processo di riga 0, che è l'unico a possedere inizialmente quella fetta, e la reduce di Y deve arrivare al processo di colonna 0, che è il proprietario designato di quella porzione del risultato.

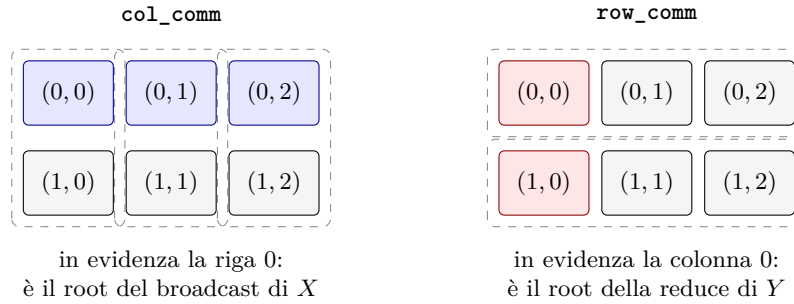


Figura 3.2: La stessa griglia, raggruppata nei due modi. Fissando la coordinata di riga si ottengono i comunicatori `row_comm`, fissando quella di colonna i comunicatori `col_comm`. Ogni processo appartiene a esattamente uno di ciascun tipo, e i due si intersecano soltanto in lui.

Indicare il root a una collettiva significa però indicarne il *rank nel sotto-comunicatore*, non le coordinate nella griglia. Il codice sfrutta allora una proprietà di `MPI_Cart_sub`: in un sotto-comunicatore con una sola dimensione residua i rank risultano ordinati per coordinata crescente. Ne segue che il rank 0 di `col_comm` è il processo di riga 0, e il rank 0 di `row_comm` è il processo di colonna 0: in entrambi i casi il root cercato, e le due collettive possono usare semplicemente 0.

La proprietà viene comunque **verificata** in `grid_create`, confrontando il rank in ciascun sotto-comunicatore con la coordinata cartesiana corrispondente. Il motivo è la libertà concessa da `reorder = 1`: se l'ipotesi cadesse, le collettive continuerebbero a funzionare regolarmente, ma con il root sbagliato, e il programma produrrebbe risultati errati senza segnalare nulla.

3.4 Partizionamento degli indici

Decidere la forma della griglia non basta: bisogna ancora stabilire, per ciascuna delle due dimensioni, quali indici globali finiscano su quale parte. La regola è una sola e viene applicata due volte — alle righe di A fra le P_r righe della griglia, alle colonne fra le P_c colonne — ed è una partizione **a blocchi**: ogni parte riceve un intervallo contiguo di indici, e il resto della divisione viene distribuito un elemento per volta sulle prime parti.

Il blocco contiguo non è una scelta di comodo. L'alternativa naturale sarebbe una distribuzione ciclica, che assegna gli indici a turno; ma qui il lavoro per elemento è uniforme, quindi il ciclico non bilancerebbe nulla che il blocco già non bilanci, e in cambio distruggerebbe la contiguità — cioè proprio la proprietà da cui dipendono sia il datatype derivato della distribuzione (sezione 3.6) sia il passo unitario degli accessi dello schema A (sezione 2.3).

3.4.1 Le funzioni

Posti $q = \lfloor n/p \rfloor$ e $r = n \bmod p$, le prime r parti hanno $q + 1$ elementi e le restanti q . Da qui le prime due funzioni, che dicono *quanti* elementi tocchino alla parte i e *da dove* cominci:

$$\text{size}(n, p, i) = q + \begin{cases} 1 & i < r \\ 0 & \text{altrimenti} \end{cases} \quad \text{start}(n, p, i) = i q + \min(i, r). \quad (3.1)$$

Il $\min(i, r)$ nell'offset conta quante parti *lunghe* precedono la parte i : finché $i < r$ sono tutte lunghe e se ne accumula una per ciascuna, dopo il loro numero resta fermo a r . La funzione è definita anche per $i = p$, dove vale n : una sentinella che rende l'ultima parte un caso come gli altri.

Lo sbilanciamento massimo è di un elemento per dimensione: fra due blocchi locali qualsiasi c'è al più una riga e una colonna di differenza.

3.5 Layout dei blocchi locali

Applicando la partizione alle due dimensioni si ottiene la forma dei tre blocchi che ogni processo possiede:

$$A_{\text{loc}} : m_{\text{loc}} \times n_{\text{loc}}, \quad X_{\text{loc}} : n_{\text{loc}} \times k, \quad Y_{\text{loc}} : m_{\text{loc}} \times k, \quad (3.2)$$

con $m_{\text{loc}} = \text{size}(M, P_r, \text{my_row})$ e $n_{\text{loc}} = \text{size}(N, P_c, \text{my_col})$.

3.5.1 Perché i conteggi delle collettive tornano

Nelle due formule qui sopra c'è una proprietà che vale la pena rendere esplicita, perché è la condizione di consistenza delle due collettive: m_{loc} dipende **solo** da `my_row` e n_{loc} **solo** da `my_col`.

Ne segue che tutti i processi di una stessa riga della griglia concordano su m_{loc} , e tutti quelli di una stessa colonna concordano su n_{loc} . Sono esattamente i due gruppi su cui agiscono le collettive: il broadcast lungo `col_comm` usa un conteggio di $n_{\text{loc}} \cdot k$ elementi, e i suoi partecipanti condividono la colonna, dunque n_{loc} ; la reduce lungo `row_comm` usa $m_{\text{loc}} \cdot k$, e i suoi partecipanti condividono la riga, dunque m_{loc} . Nessuno dei due conteggi va negoziato o comunicato: viene dalla stessa formula, valutata su coordinate che i partecipanti hanno in comune.

3.5.2 Chi contiene che cosa, e quando

Il buffer X_{loc} esiste su *tutti* i processi — il broadcast deve avere dove scrivere — ma prima del broadcast contiene dati significativi soltanto sui processi della riga 0. Dopo il prodotto locale ogni processo possiede un contributo *parziale* a Y , che riguarda le proprie righe ma soltanto le proprie colonne di A ; la `MPI_Reduce` lungo `row_comm` li somma e produce il blocco definitivo sui processi della colonna 0.

3.5.3 Perché il multivettore non viene mai tagliato lungo k

Le k colonne restano indivise su tutti i processi. Tre ragioni, in ordine di peso:

- **costringerebbe a replicare A .** Per calcolare un qualunque sottoinsieme di colonne di Y serve comunque *tutta* A : i processi di una stessa partizione lungo k vorrebbero perciò lo stesso identico blocco di A . Si duplicherebbe il dato dominante (??), che è l'opposto di ciò per cui la decomposizione esiste;
- **ridurrebbe il riuso di A in registro.** Nello schema A il fattore di riuso è pari al numero di colonne che il processo possiede localmente (sezione 2.3): frazionare k lo frazionerebbe nella stessa misura, abbassando l'intensità aritmetica del nucleo;
- **produrrebbe messaggi troppo piccoli.** k è piccolo per definizione del problema, e frammentarlo ulteriormente darebbe comunicazioni dominate dalla latenza invece che dalla banda.

Invariante critico

Il numero di righe locali di X deve venire dalla *stessa* partizione con cui si tagliano le colonne di A , e il numero di righe locali di Y dalla stessa con cui se ne tagliano le righe. Non è un controllo da eseguire, è una proprietà garantita *per costruzione*: esistono soltanto m_{loc} e n_{loc} , calcolati una volta sola in `layout_init`, e non due coppie di valori da tenere allineate. La struttura `layout_t` è il punto unico di verità sulla forma dei blocchi, e finché resta tale l'invariante non può essere violato senza modificarla.

3.6 Distribuzione iniziale dei dati

Le modalità di inizializzazione di A e X sono indipendenti (`-a-mode`, `-x-mode`), e tutte e quattro le combinazioni sono valide. Le due modalità di ciascuna matrice producono gli stessi elementi globali a parità di seed, per la proprietà del generatore discussa in ??.

-a-mode global. Il grid rank 0 materializza A completa e la distribuisce. È il caso interessante dal punto di vista MPI: i sottoblocchi 2D **non sono contigui** nella matrice row-major globale. Il mittente descrive il blocco da inviare con un `MPI_Type_vector` di m_{loc} blocchi da n_{loc} elementi e stride N . Il destinatario ne descrive uno con stride `lda`, che può essere diverso: è il caso generale - due layout differenti sui due lati della stessa comunicazione - e non richiede nessun buffer intermedio né nessuna copia manuale. Il rank 0 fa eccezione per il proprio blocco, che copia direttamente senza passare da MPI.

-a-mode local. Ogni processo genera direttamente il proprio blocco 2D di A dagli indici globali. In questo modo non è necessario materializzare una copia globale aggiuntiva della matrice su un singolo processo.

-x-mode global Il grid rank 0 materializza la X compatta $N \times k$ e un `MPI_Scatterv` sul `row_comm` della riga 0 ne distribuisce i blocchi di righe: entrambi i lati sono contigui, perché $ldx = k$ ovunque, quindi non serve nessun datatype derivato.

-x-mode local I processi della riga 0 generano direttamente le proprie righe di X . Il successivo broadcast lungo `col_comm` è identico alla modalità `global`.

Generazione e distribuzione iniziale fanno parte del **preprocessing**, perciò sono escluse dai tempi ufficiali. La modalità `global` rappresenta uno scenario in cui l'input è inizialmente centralizzato.

3.7 L'algoritmo distribuito

Un'invocazione del prodotto esegue tre passi:

```
MPI_Bcast(X_loc, n_loc*k, SCALAR_MPI_TYPE, 0, grid->col_comm);

local_gemm(local_gemm_context, X_loc, layout->ldx, Y_loc_part, layout->ldy);

MPI_Reduce(Y_loc_part, Y_row_col0, m_loc*k, SCALAR_MPI_TYPE,
           MPI_SUM, 0, grid->row_comm);
```

Listato 3.1: Il corpo di `mpi_matmul`, senza i cronometri.

1. **Broadcast di X lungo `col_comm`:** tutti i processi di una colonna della griglia lavorano sulle stesse colonne di A , quindi servono loro le stesse righe di X . Il root è il processo di riga 0, l'unico che possiede inizialmente quella fetta, ne è il *responsabile*.
2. **Prodotto locale:** il risultato è *parziale*: il processo in posizione (r, c) calcola il contributo delle sole colonne $[col_0, col_0 + n_{loc})$ di A alle righe $[row_0, row_0 + m_{loc})$ di Y (dove row_0 e col_0 sono gli offset globali del blocco locale).
3. **Reduce lungo `row_comm`:** i P_c contributi parziali della stessa banda di righe vivono sui processi di una stessa riga della griglia: sommandoli tra loro si ottiene Y . Il count è $m_{loc} \cdot k$ (e non 1) dato che la riduzione agisce lungo la dimensione dei processi e preserva la posizione, producendo dunque $m_{loc} \cdot k$ somme indipendenti. Il root è il processo di colonna 0, che è già il *responsabile* designato di quella porzione di Y : nessuna ridistribuzione finale è necessaria.

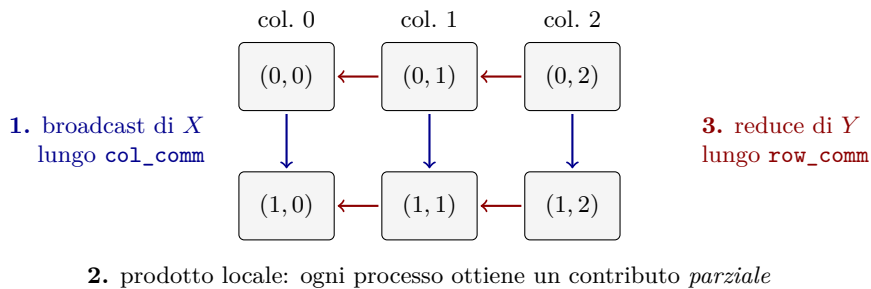


Figura 3.3: Le due collettive di una singola invocazione. Il broadcast fa scendere la fetta di X dal processo di riga 0 a tutti i processi della sua colonna, che ne hanno bisogno perché condividono le stesse colonne di A . Dopo il prodotto locale ogni processo possiede un contributo *parziale* a Y : la reduce somma i P_c parziali di una stessa banda di righe e deposita il risultato sul processo di colonna 0, che ne è il proprietario designato.

3.8 Modello di costo della comunicazione

Il costo di un messaggio di n byte fra due processi si modella, seguendo le dispense del corso, come

$$T(n) = \lambda + \frac{n}{B}, \quad (3.3)$$

dove λ è la *latenza*, un costo fisso che si paga per ogni messaggio indipendentemente dalla sua taglia, e B è la *banda*. In letteratura lo stesso modello compare spesso nella forma $\alpha + \beta n$: il dizionario è $\alpha \equiv \lambda$ e $\beta \equiv 1/B$.

Le due collettive non sono però messaggi singoli. Il costo logaritmico che segue è quello del *recursive doubling*, l'algoritmo di broadcast visto a lezione, che al posto dei $p-1$ invii sequenziali della versione ingenua costa $T(p) = \lceil \log_2 p \rceil$ tempi di messaggio. Il logaritmo si ottiene grazie a un'ipotesi precisa sulla rete — che essa sostenga più messaggi contemporaneamente, purché le coppie di nodi coinvolte non si sovrappongano — ed è quell'ipotesi, più che la struttura ad albero, a rendere possibile il guadagno.

Detti s i byte per scalare, il costo di comunicazione per invocazione è quindi

$$T_{\text{comm}} \approx \underbrace{\lceil \log_2 P_r \rceil \left(\lambda + \frac{s k N / P_c}{B} \right)}_{\text{broadcast di } X} + \underbrace{\lceil \log_2 P_c \rceil \left(\lambda + \frac{s k M / P_r}{B} + \gamma k \frac{M}{P_r} \right)}_{\text{reduce di } Y}, \quad (3.4)$$

dove γ è il costo per elemento della somma eseguita dalla reduce. Il termine in γ non compare nel modello del corso, che descrive il costo del solo trasferimento: lo aggiungiamo perché la reduce, a differenza del broadcast, a ogni livello dell'albero esegue anche $m_{\text{loc}} \cdot k$ addizioni.

Il modello va letto per quello che è: i fattori logaritmici valgono se l'implementazione usa il recursive doubling, ma le librerie MPI cambiano algoritmo al crescere del messaggio e per messaggi grandi passano a schemi a pipeline o scatter/allgather, nei quali il termine di banda perde il fattore logaritmico e resta proporzionale al solo volume trasferito.

Il modello va letto per quello che è: i fattori logaritmici valgono se l'implementazione usa il recursive doubling, ma — come le dispense stesse osservano — una buona libreria MPI commuta internamente fra algoritmi diversi a seconda dell'operazione, della rete e soprattutto della quantità di dati. Per messaggi grandi si passa a schemi a pipeline o scatter/allgather, nei quali ogni byte attraversa la rete una volta sola invece di $\log_2 p$ volte: il termine di banda perde il fattore logaritmico e resta proporzionale al solo volume, al prezzo di un numero maggiore di messaggi e quindi di più latenza.

Alle taglie della campagna i messaggi cadono in quel regime, quindi il $\log_2$ va inteso come limite superiore sui termini di latenza più che come andamento atteso della banda. Questo non intacca però la conclusione che segue: il *volume*, cioè l'unica quantità da cui dipende la scelta della forma della griglia, è lo stesso per qualunque algoritmo collettivo, perché è quanto ciascun processo deve comunque ricevere o inviare.

Il termine che decide la forma ottima della griglia è il volume:

$$V(P_r, P_c) = k \left(\frac{N}{P_c} + \frac{M}{P_r} \right), \quad P_r P_c = P. \quad (3.5)$$

Minimizzando con il vincolo $P_r P_c = P$ si ottiene

$$P_r^* = \sqrt{P \frac{M}{N}}, \quad P_c^* = \sqrt{P \frac{N}{M}}, \quad V^* = \frac{2k\sqrt{MN}}{\sqrt{P}}. \quad (3.6)$$

Due letture, entrambe verificate sperimentalmente in ??:

- **Perché 2D e non 1D.** Le due decomposizioni monodimensionali sono i casi degeneri $P_r = 1$ (volume kM , nessun broadcast ma reduce su tutti i P processi) e $P_c = 1$ (volume kN , nessuna reduce ma broadcast su tutti i P). In entrambi il volume *non decresce* con P , mentre la griglia 2D lo fa scendere come $1/\sqrt{P}$. È il motivo per cui la traccia richiede una griglia bidimensionale, ed è ciò che la campagna C6 misura direttamente confrontando tutte le fattorizzazioni di P , forme degeneri comprese.
- **Perché la forma ottima dipende da $M:N$.** Per $M = N$ la griglia migliore è quella quadrata, cioè proprio quella che `grid_default_shape` sceglie da sé — ed è anche quella che lo standard MPI sceglierebbe da sé, visto che `MPI_Dims_create` costruisce la griglia più vicina possibile alla forma quadrata. La eq. (3.6) dice però che quel default è ottimo *soltanto* per $M = N$; per $M = 3N$ conviene invece $P_r > P_c$. La campagna C1 varia il rapporto $M:N$ a lavoro costante e rende visibile questo effetto separandolo da quello della taglia.

Un'osservazione che va tenuta presente leggendo il weak scaling (campagna C5): su una griglia 2D con blocco locale costante, M segue P_r e N segue P_c , quindi cambiare P cambia *anche* il rapporto $M:N$. È inerente alla decomposizione, ed è la ragione per cui l'effetto della forma viene misurato separatamente in C1 e C6.

Capitolo 4

Il nucleo di CPU

Il nucleo di CPU non è la parte più veloce del progetto, ma è quella di cui tutto il resto ha bisogno per essere giudicato: è il denominatore di ogni speedup, ed è l'unica implementazione che gira ovunque, anche su una macchina senza `nvcc`. Merita quindi lo stesso lavoro dei backend CUDA, e per le stesse ragioni.

4.1 Tre backend, una sola idea

Tutti e tre i backend di CPU implementano lo schema A di sezione 2.3: cambia come il lavoro viene organizzato attorno a quell'ordine dei cicli, non l'ordine. Si scelgono a tempo di compilazione con la variabile `KERNEL` del `Makefile`, espongono la stessa interfaccia (??) e i binari coesistono, così un confronto è una sola campagna sugli stessi dati.

- **scheme_a**: l'implementazione di riferimento, quella che corrisponde alla consegna;
- **scheme_a_jblock**: l'implementazione di riferimento è la stessa del caso precedente, ma con X in cui è stato applicato un meccanismo di *tiling* in cache;
- **scheme_a_jblock_rb**: come sopra, ma all'implementazione è stato aggiunto il *register blocking* sulle righe di A .

I tre non sono tre implementazioni alternative, ma riflettono la **catena** di sviluppo: ciascuno rimuove il collo di bottiglia che il precedente lascia scoperto. Il capitolo li presenta in quest'ordine, e ogni sezione si apre con il limite che motiva il backend successivo.

4.2 Il backend di riferimento

4.2.1 Perché specializzare k a compile-time

Nello schema A i k accumulatori sono variabili locali del ciclo su j , e tutto il vantaggio dello schema dipende dal fatto che questi accumulatori vivano nei **registri**: è lì che il valore $A[i][j]$, letto una volta sola, può essere riutilizzato k volte.

Affinché il compilatore possa rappresentare gli accumulatori come un insieme fisso di valori scalari candidati a vivere nei registri, il numero di accumulatori deve essere *noto a compile-time*. Con un array dimensionato dinamicamente, invece, l'accesso `acc[c]` conserva un indice runtime e il compilatore non dispone della stessa struttura esplicita.

Se il numero di accumulatori è noto solo quando il programma parte, la variabile `acc` diventa un array in memoria e ogni operazione di accumulazione (`acc[c] += ...`) è un accesso a quella memoria: esattamente il traffico che lo schema A serviva a evitare.

Il codice risolve generando **cinque funzioni specializzate**, per $k \in 3, 6, 8, 20, 32$, cioè per i cinque valori richiesti dalla traccia, a partire da una sola definizione via macro:

```
#define COLS_8(M) COLS_6(M) M(6) M(7)
#define DECLARE_ACC(c) scalar_t acc##c = (scalar_t)0;
#define UPDATE_ACC(c) acc##c += a * xrow[c];
#define STORE_ACC(c) yrow[c] = acc##c;

/* per ogni riga i: */
COLS(DECLARE_ACC)
for (j = 0; j < n_loc; j++) {
    const scalar_t a = arow[j];          /* letto UNA volta */
    const scalar_t *restrict xrow = X_loc + (size_t)j * ldx;
    COLS(UPDATE_ACC)                     /* k aggiornamenti espliciti */
}
COLS(STORE_ACC)
```

Listato 4.1: Generazione delle specializzazioni (estratto di `src/kernel/scheme_a.c`).

Vale la pena leggere che cosa producono queste righe: `COLS_8` è un elenco di indici, applicarlo a `DECLARE_ACC` genera 8 dichiarazioni di variabili scalari *distinte* (`acc0`, `acc1`, ...), non un array, mentre applicarlo a `UPDATE_ACC` genera 8 aggiornamenti scritti per esteso, senza alcun ciclo interno con estremo runtime.

L'hot path che il compilatore vede non contiene quindi nessuna indicizzazione su `c`: contiene k istruzioni indipendenti su k variabili indipendenti, che fornisce al compilatore una forma particolarmente favorevole all'allocazione degli accumulatori nei registri e alla successiva vettorizzazione.

La generazione effettiva di codice vettoriale e l'eventuale uso di FMA sono proprietà del codice macchina prodotto dal compilatore e vengono quindi verificate separatamente tramite profilazione.

In questo contesto, vettorizzare un ciclo significa che il compilatore riesce a trasformare le k istruzioni indipendenti in un'unica istruzione vettoriale, che opera su W scalari per volta, dove W è la larghezza del registro vettoriale della CPU, utilizzando istruzioni SIMD. Se k è minore di W , il compilatore può comunque generare un'istruzione vettoriale, ma non sarà completamente utilizzata. Se k è maggiore di W , il compilatore genererà più istruzioni vettoriali per coprire tutti gli accumulatori.

4.2.2 Il fallback generico, e perché la traccia lo richiede

La traccia chiede però che il codice funzioni per k *generico*, e cinque specializzazioni non possono coprire tutti i valori possibili. Per ogni altro k interviene un **fallback generico**, che non tenta di tenere k accumulatori distinti, perché sarebbero troppi e finirebbero in memoria, ma lavora a gruppi di al più $KB = 32$ colonne per volta. Ogni gruppo richiede una passata completa su A , con un numero di passaggi pari a:

$$\text{passaggi su } A = \left\lceil \frac{k}{KB} \right\rceil. \quad (4.1)$$

Per $k \leq KB$ il ciclo esterno sui gruppi è *degenere*: esiste nel codice ma gira una volta sola, A viene letta una volta sola e il fallback si differenzia dalle specializzazioni soltanto per l'array di accumulatori invece delle variabili distinte. Per $k > 32$ il numero di passaggi cresce secondo $\lceil k/32 \rceil$, ad esempio tra 33 e 64 sono necessari due passaggi, tra 65 e 96 tre, e così via.

La conseguenza va tenuta presente leggendo i risultati: le righe dei k oltre la soglia e quelle dei k entro la soglia **non sono direttamente confrontabili**, perché non stanno facendo la stessa quantità di lavoro di memoria.

4.3 Perché $KB = 32$

Il valore copre tutti i k del collaudo, ma questa da sola non è una ragione sufficiente. La ragione vera è un budget di registri.

Infatti con KB accumulatori il compilatore deve tenere KB valori vivi attraverso l'intero ciclo su j . Vettorizzati, occupano KB/W registri vettoriali, dove W è il numero di scalari che entrano in un registro, a fronte di 16 registri vettoriali sulle configurazioni AVX2 e 32 sulle configurazioni AVX-512. Al crescere di KB il riuso di A migliora, perché i passaggi della eq. (4.1) diminuiscono, ma cresce la pressione sui registri fino al fenomeno del *register spilling*, cioè fino al punto in cui il compilatore è costretto a parcheggiare in memoria gli accumulatori che non stanno più nei registri e a quel punto si va a perdere più di quanto si guadagni.

Il valore 32 è quindi il compromesso scelto dall'implementazione: è sufficiente a coprire in un solo passaggio tutti i 5 k richiesti dalla traccia, ma mantiene il numero di accumulatori entro una dimensione compatibile con un'implementazione a registri. Il fatto che tale scelta eviti effettivamente lo spilling e sia ottimale sulla macchina di riferimento non è assunto come proprietà teorica: viene verificato sperimentalmente nella campagna C3 (??).

4.4 Ciò che il compilatore vede

Affinché questo basti, il codice deve rispondere da sé alle tre domande che un compilatore si pone prima di vettorizzare un ciclo.

- «**Questi puntatori possono riferirsi alla stessa memoria?**» No: A , X e Y sono dichiarati **restrict**. La garanzia va però *ripristinata* dentro `local_gemm`, con copie locali dei puntatori: il contesto opaco tiene il puntatore ad A come membro di una struttura, e senza quella copia i kernel specializzati perderebbero l'informazione di non aliasing che avevano quando A era un parametro. Se l'aliasing fosse possibile, ogni scrittura su Y potrebbe invalidare A e il ciclo andrebbe eseguito in ordine.
- «**Le iterazioni dipendono l'una dall'altra?**» No: i k aggiornamenti `acc[c] += a * X[j][c]` riguardano accumulatori distinti e sono indipendenti fra loro. Essendo già srotolati dalle macro, si presentano come catene di moltiplicazione-accumulo indipendenti, che è la forma che una unità FMA può eseguire in parallelo.
- «**Gli accessi sono regolari?**» Sì, e a passo unitario: come osservato in sezione 2.3, sia la riga di A sia la riga di X sono contigue in memoria.

Da completare con i dati della campagna

Verificare sul server l'effettiva vettorizzazione del percorso caldo, per esempio con `gcc -fopt-info-vec` oppure ispezionando l'assembly di `kernel_k8` e `kernel_k32`. Il punto da documentare è se e a quale k compaiano spill degli accumulatori.

4.5 Il primo limite: la rilettura di X

Lo schema A legge A una volta sola, ed è il suo pregio. Ma per ogni riga i di A percorre **tutta** X , e questo è il suo prezzo: il traffico in lettura su X vale

$$\text{elementi di } X \text{ letti} = m_{\text{loc}} \cdot n_{\text{loc}} \cdot k, \quad (4.2)$$

cioè k volte il traffico su A .

Finché X resta in una cache vicina al calcolo questo traffico non raggiunge la memoria centrale e semplicemente non si vede. Quando X non ci sta più, viene ristreamata da un livello più lontano una volta per ogni riga di A , e il tempo per elemento smette di essere governato dal calcolo per essere governato dalla banda di quel livello.

La soglia è esplicita e dipende da entrambi i parametri del problema: il blocco locale X_{loc} occupa $n_{\text{loc}} \cdot k \cdot s$ byte. A parità di matrice, quindi, è k **che decide da che parte della soglia ci si trova**, ed è per questo che l'effetto si manifesta come un calo dei GFLOPS al crescere di k — cioè nella direzione opposta a quella che l'intensità aritmetica della ?? lascerebbe prevedere. È la chiave di lettura dei risultati di ??, ed è il limite che il backend successivo rimuove.

4.6 Secondo backend: X mattonellata in cache

L'idea di `scheme_a_jblock` è di non percorrere più tutta X per ogni riga di A , ma di prenderne un **tile** — un blocco di righe consecutive — e di applicarlo a tutte le righe di A prima di passare al successivo:

```
for j0 (tile di X di BJ righe):
  for i (TUTTE le righe di A):
    acc = (primo tile) ? 0 : Y[i][*]
    for j nel tile: a = A[i][j]; acc[c] += a * X[j][c]
    Y[i][*] = acc
```

Listato 4.2: Struttura di `src/kernel/scheme_a_jblock.c`.

Il tile viene caricato dalla memoria una volta e riusato m_{loc} volte restando in cache. A continua a essere letta *una volta sola*: ogni sua riga viene semplicemente consumata a segmenti, uno per tile, invece che tutta di seguito.

Il prezzo. Y non può più vivere nei registri per tutta la durata di una riga, perché ci si torna sopra a ogni tile: va riletta e riscritta ogni volta. Il traffico aggiuntivo vale $2k$ scalari per riga e per tile, cioè un rapporto $2k/\text{BJ}$ rispetto al traffico su A . Il tile va quindi scelto *grande* abbastanza da rendere questo termine trascurabile e *piccolo* abbastanza da stare in cache: le due richieste tirano in direzioni opposte, e il punto di equilibrio non è deducibile a priori.

Per questo la dimensione del tile è un parametro di build, `JBLOCK_BYTES`, e non una costante scelta una volta per tutte: il codice ne ricava il numero di righe per tile dividendo per $k \cdot s$, il default è un valore che sta nella cache di secondo livello di qualunque core x86 recente, e quale sia il valore migliore è oggetto di misura.

Un principio già visto. È esattamente la stessa mossa del backend CUDA con tile in shared memory (??): portare un blocco di X in una memoria vicina al calcolo e riusarlo per molte righe prima di passare al blocco successivo. Cambia il nome della memoria vicina — lì la shared memory, qui la cache — non il ragionamento.

4.7 Terzo backend: register blocking sulle righe

Tolto il limite di banda, ne resta scoperto un altro, che si manifesta ai k *piccoli*: in tutti i backend visti finora ogni `acc[c]` è una catena *seriale* di moltiplicazioni-accumulo lungo j — ogni aggiornamento deve attendere il precedente — e l'unico parallelismo a livello di istruzione disponibile è quello fra le k colonne. Poiché una unità FMA ha una latenza di diversi cicli e

la CPU ne ha più di una, per tenerle occupate servono più catene indipendenti di quante k piccolo ne offra: il processore aspetta, e i GFLOPS crescono con k invece di essere costanti.

`scheme_a_jblock_rb` risolve lavorando su RB righe di A per volta:

```
for j0 (tile di X, come in scheme_a_jblock):
  for i a passi di RB:
    acc[r][c] = (primo tile) ? 0 : Y[i+r][c]
    for j nel tile:
      x[c] = X[j][c] /* caricato UNA volta */
      for r: a = A[i+r][j]; acc[r][c] += a * x[c] /* riusato RB volte */
    Y[i+r][c] = acc[r][c]
```

Listato 4.3: Struttura di `src/kernel/scheme_a_jblock_rb.c`.

Le righe di A sono indipendenti per costruzione — nessun elemento di Y dipende da un'altra riga — quindi il numero di catene indipendenti passa da $\lceil k/W \rceil$ a $RB \cdot \lceil k/W \rceil$. In più, ogni valore di X caricato viene riusato RB volte direttamente dai registri, quindi anche il traffico su X — già in cache grazie al tile — si riduce dello stesso fattore.

Chi sceglie RB. Anche qui il limite sono i registri: $RB \cdot \lceil k/W \rceil$ accumulatori più RB valori di A devono starci tutti, altrimenti si torna allo spill e si perde più di quanto si guadagna. Il codice sceglie perciò RB *a tempo di compilazione e per ogni k* , dal numero di registri vettoriali e dalla loro ampiezza dell'architettura di destinazione (`VEC_NREGS` e `VEC_W`, fissati dalle macro del compilatore), prendendo il più grande fra i valori ammessi che soddisfi il vincolo; la variabile di build `RB_ROWS` permette di forzarlo per misurarne l'effetto. Le righe che avanzano quando m_{loc} non è multiplo di RB sono chiuse dal kernel a una riga, che è il caso $RB = 1$ dello stesso schema.

I tre backend rispondono quindi a tre limiti diversi e successivi: il traffico su A , risolto dallo schema A stesso; la banda verso X , risolta dal tile in cache; il parallelismo di istruzione, risolto dal blocco di righe. Quanto ciascuno dei tre passi valga davvero sulla macchina di misura è ciò che la campagna riportata in ?? quantifica.

Capitolo 5

I backend CUDA

L'implementazione su GPU è stata sviluppata attraverso tre kernel CUDA, affiancati da un backend basato su cuBLAS, usato come riferimento esterno. Le diverse versioni non sono state progettate come soluzioni indipendenti: partono dalla stessa operazione locale, $Y = AX$, e introducono progressivamente modifiche mirate alla mappatura del lavoro e agli accessi alla memoria.

In particolare, il percorso seguito è il seguente:

- **cuda_naive** costituisce la baseline CUDA. A ciascun thread viene assegnato un singolo elemento Y_{ic} : il thread fissa quindi una riga i di A e una colonna c di X , e percorre l'intera dimensione interna j per calcolarne il prodotto scalare. La mappatura è semplice e offre un primo termine di confronto, ma non sfrutta il riuso della riga di A tra le k colonne del multivettore.
- **cuda_warp** cambia la granularità della parallelizzazione: non più un thread per elemento di Y , ma un *warp* per un'intera riga di Y . Le 32 lane del warp si suddividono gli elementi della corrispondente riga di A e ciascuna lane riutilizza il valore di A appena caricato per aggiornare i k risultati della riga. Al termine, i contributi prodotti dalle lane vengono sommati mediante primitive di warp shuffle.
 - **cuda_warp** nella variante **xcol** mantiene esattamente la stessa organizzazione del calcolo, ma modifica la rappresentazione di X . Il blocco locale del multivettore viene convertito una sola volta, durante il preprocessing, da una disposizione in memoria per righe (*row-major*) a una disposizione per colonne (*column-major*). In questo modo quando le lane del warp avanzano su indici j consecutivi, accedono anche a elementi consecutivi di X .
 - **cuda_warp_smem** affronta lo stesso problema degli accessi a X senza però andarne a modificare il layout. Un blocco di thread carica *cooperativamente* una porzione di X dalla memoria globale alla **shared memory**. I warp appartenenti allo stesso blocco possono quindi riutilizzare quella stessa porzione durante il calcolo delle rispettive righe di Y .

Il passaggio da **cuda_naive** a **cuda_warp** modifica la mappatura del lavoro e introduce il riuso di A all'interno del warp. A questo punto è emerso un problema: con X nel normale layout row-major, le lane che avanzano lungo j non leggono elementi consecutivi del multivettore.

La variante **xcol** interviene sulla *rappresentazione del dato*, convertendo preventivamente X in un layout più favorevole agli accessi del warp, mentre **cuda_warp_smem** conserva il layout originario e interviene sul *percorso del dato*, caricando e riutilizzando X attraverso la shared

memory. Le due versioni permettono così di studiare due strategie diverse per lo stesso collo di bottiglia, senza sovrapporle nello stesso kernel.

Infine, si propone anche un'implementazione su GPU basata su cuBLAS, la libreria NVIDIA per algebra lineare ottimizzata per GPU:

- **cublas** non rappresenta un'ulteriore ottimizzazione del kernel sviluppato nel progetto. È un riferimento esterno, inserito dietro la stessa interfaccia degli altri backend, che permette di confrontare le implementazioni precedenti con una libreria GPU general-purpose sulla stessa macchina e nelle stesse condizioni di misura.

5.1 Struttura comune

Tutti i backend GPU rispettano lo stesso ciclo di vita a tre fasi introdotto in ??, che permette di confrontare le diverse implementazioni misurando, in ciascun caso, la stessa porzione dell'esecuzione.

La distinzione fondamentale deriva dal diverso ruolo dei dati. Il blocco locale di A non cambia tra un'invocazione del prodotto e la successiva e può quindi essere trasferito sulla GPU una sola volta. Al contrario, la fetta locale di X deve essere resa disponibile a ciascun processo a ogni invocazione del prodotto: le sue porzioni sono inizialmente collocate sulla prima riga della griglia di processi e vengono replicate lungo ciascuna colonna mediante `MPI_Bcast.`, mentre il risultato parziale Y deve essere riportato sull'host prima della successiva `MPI_Reduce.` Di conseguenza, le operazioni comuni ai backend CUDA sono distribuite nelle tre fasi nel modo seguente:

- **local_gemm_create** prepara il backend prima dell'inizio delle ripetizioni cronometrate. In questa fase viene inizializzato il contesto CUDA, vengono allocati in VRAM i buffer destinati ad A , X e Y , vengono creati gli eventi CUDA usati per le misure e il blocco locale di A viene copiato dall'host al device. Poiché A rimane invariata per tutte le invocazioni successive, questo trasferimento viene eseguito una sola volta. L'intera fase appartiene al preprocessing e non entra nel tempo ufficiale del prodotto. I suoi costi principali vengono comunque registrati separatamente come inizializzazione del device, allocazione della memoria e trasferimento H2D di A .
- **local_gemm** rappresenta invece il cammino percorso a ogni invocazione. Prima del calcolo, il nuovo blocco di X viene trasferito dall'host alla GPU. Segue l'esecuzione del kernel CUDA, oppure della chiamata cuBLAS nel backend di riferimento. Infine il risultato parziale Y viene copiato nuovamente verso l'host. Il tempo `t_local` comprende quindi l'intera sequenza H2D di X , calcolo e D2H di Y .

All'interno della stessa funzione, tre coppie distinte di `cudaEvent` delimitano separatamente il trasferimento H2D di X , il solo calcolo sul device e il trasferimento D2H di Y . Si ottengono in questo modo tre misurazioni: rispettivamente $t_{H2D X}$, t_{kernel} e $t_{D2H Y}$. Questa separazione è necessaria perché la specifica del progetto richiede di escludere dal tempo ufficiale i trasferimenti da e verso la scheda: per i backend CUDA è quindi t_{kernel} , e non l'intero t_{local} , a rappresentare il contributo del calcolo locale alla misura prestazionale.

- **local_gemm_destroy** conclude infine il ciclo di vita del backend, liberando i buffer allocati sulla GPU, gli eventi CUDA e il contesto mantenuto dall'implementazione. Anche questa fase avviene al di fuori delle ripetizioni misurate.

La stessa suddivisione viene mantenuta in tutti i backend GPU. Cambia quindi il modo in cui viene eseguito il prodotto all'interno della seconda fase, ma non cambiano né la posizione

dei trasferimenti né i confini delle misure. Il confronto tra i kernel può così essere effettuato sul solo t_{kernel} , mentre t_{local} rimane disponibile per osservare anche il costo reale dell'intera invocazione lato GPU.

5.2 Misurazione del backend CUDA

La natura asincrona dell'esecuzione CUDA richiede di distinguere il tempo osservato dall'host dal tempo trascorso effettivamente sul device. Il lancio di un kernel, infatti, restituisce il controllo alla CPU senza attendere necessariamente il completamento dell'operazione. Un cronometro host posto semplicemente prima e dopo il lancio misurerebbe semplicemente il costo della chiamata al runtime CUDA, non il tempo di esecuzione del kernel, ciò che invece ci interessa.

Per isolare le singole operazioni eseguite sul device, i backend CUDA utilizzano coppie di **cudaEvent** registrate sullo stesso stream delle operazioni misurate: in questo modo gli eventi permettono di determinare separatamente il tempo necessario al trasferimento H2D di X , all'esecuzione del calcolo e al trasferimento D2H di Y .

Il tempo della fase locale osservato dal driver, indicato con t_{local} , ha invece un significato diverso: misura l'intera invocazione del backend e comprende quindi sia il lavoro eseguito sul device sia l'overhead necessario per richiederlo e sincronizzarlo.

Schematizzando il calcolo, otteniamo:

$$t_{\text{local}} = t_{\text{H2D } X} + t_{\text{kernel}} + t_{\text{D2H } Y} + t_{\text{overhead}}, \quad (5.1)$$

dove t_{overhead} non viene misurato direttamente, ma rappresenta il residuo fra il tempo complessivo osservato dall'host e i tre intervalli misurati mediante eventi CUDA. Esso può comprendere il costo delle chiamate al runtime, delle sincronizzazioni e della gestione della strumentazione.

La distinzione è rilevante anche per la valutazione delle prestazioni, perché come richiesto dalla specifica del progetto, i trasferimenti tra host e device non devono contribuire al tempo utilizzato per calcolare i FLOPS. Per i backend CUDA il contributo computazionale è quindi t_{kernel} , mentre $t_{\text{H2D } X}$ e $t_{\text{D2H } Y}$ vengono conservati come misure separate e analizzati successivamente.

Oltre al tempo, il backend comunica al driver anche il numero di byte trasferiti nelle tre operazioni principali: la copia iniziale di A , la copia di X eseguita a ogni invocazione e la copia di ritorno di Y . Tempo e volume trasferito permettono di ricavare una **banda effettiva**, calcolata come:

$$B_{\text{eff}} = \frac{D}{t}, \quad (5.2)$$

dove D è il numero di byte trasferiti e t il tempo della corrispondente copia.

Il solo tempo di trasferimento sarebbe infatti poco informativo: una copia più grande richiede naturalmente più tempo di una più piccola. La banda effettiva permette invece di confrontare trasferimenti di dimensioni differenti e di valutare quanto efficacemente venga utilizzato il collegamento tra host e GPU.

5.3 Preprocessing e occupazione della GPU

Anche il costo di creazione del backend viene registrato, pur rimanendo esterno alla regione utilizzata per la misura ufficiale. Il tempo t_{setup} viene distinto in tre contributi, perché corrispondono a operazioni di natura differente.

Il primo è l'inizializzazione dell'ambiente CUDA, necessaria affinché il processo possa utilizzare il device. Si tratta principalmente di un costo di avvio e non dipende direttamente dalle dimensioni del prodotto da calcolare.

Il secondo contributo riguarda l'allocazione in memoria device dei buffer necessari per A , X e Y . Questo costo è legato alla preparazione delle risorse del backend e non rappresenta un trasferimento di dati.

Il terzo contributo è invece la copia H2D del blocco locale di A . Poiché A rimane invariata durante tutte le successive invocazioni del prodotto, questa copia viene effettuata una sola volta in `local_gemm_create`. Il suo costo viene quindi escluso dal cammino cronometrato, ma misurato separatamente.

Questa distinzione permette di quantificare direttamente il vantaggio della persistenza di A sul device: se il suo trasferimento fosse ripetuto a ogni chiamata di `local_gemm`, il relativo costo entrerebbe nel percorso critico insieme alla copia di X e a quella di Y .

5.3.1 Gestione della memoria disponibile

La disponibilità della memoria device non viene verificata mediante una stima preventiva utilizzata per decidere se procedere con il calcolo. Il backend tenta invece direttamente le allocazioni necessarie mediante `cudaMalloc` e, a questo punto, se si verifica un errore di allocazione, viene registrato un messaggio di errore e si procede con il fallimento della creazione del backend.

Questa scelta evita di basare la decisione su una precedente interrogazione della memoria libera: infatti, in presenza di più processi MPI che condividono la stessa GPU, il valore restituito da una precedente osservazione dello stato della memoria può cambiare prima dell'allocazione effettiva. In caso di errore, `cudaMemGetInfo` viene comunque utilizzata per produrre una diagnostica che riporta la memoria richiesta e quella disponibile.

I principali buffer persistenti richiedono complessivamente una quantità di byte pari a:

$$(m_{\text{loc}} \cdot \text{lda} + n_{\text{loc}} \cdot \text{ldx} + m_{\text{loc}} \cdot \text{ldy}) \cdot \text{sizeof}(\text{scalar_t}) \quad (5.3)$$

Poiché nel problema considerato k è piccolo rispetto alle dimensioni della matrice, il termine associato ad A è normalmente dominante.

Aumentare il numero di processi MPI e scegliere opportunamente la griglia bidimensionale riduce le dimensioni dei blocchi locali e, di conseguenza, la quantità di memoria device richiesta da ciascun processo.

5.4 `cuda_naive`

La prima implementazione CUDA adotta la mappatura più diretta possibile: a ciascun thread viene assegnato un singolo elemento del risultato locale Y .

Indicando con t l'identificativo globale del thread, gli indici dell'elemento da calcolare sono

$$i = \left\lfloor \frac{t}{k} \right\rfloor, \quad c = t \bmod k. \quad (5.4)$$

Il thread t è quindi responsabile di Y_{ic} e ne calcola il valore percorrendo interamente la dimensione interna del prodotto:

$$Y_{ic} = \sum_{j=0}^{n_{\text{loc}}-1} A_{ij} X_{jc}. \quad (5.5)$$

Ogni thread mantiene un unico accumulatore e non comunica con gli altri thread.

Il kernel costituisce per questo una baseline semplice, utile sia per verificare il corretto funzionamento dell'intera pipeline CUDA, sia come riferimento interno rispetto al quale valutare le ottimizzazioni successive.

La semplicità della mappatura comporta però una prima inefficienza fondamentale. Infatti, per una stessa riga i , gli elementi:

$$Y_{i0}, Y_{i1}, \dots, Y_{i,k-1}$$

vengono calcolati da thread distinti. Durante una stessa iterazione j , tutti questi thread hanno bisogno dello stesso coefficiente A_{ij} , ma ciascuno lo utilizza soltanto per la propria colonna di Y . Il kernel non esprime quindi alcun riuso di A_{ij} all'interno del thread: lo stesso elemento di A compare logicamente in k operazioni di caricamento differenti.

Questo è il principale limite che motiva il backend successivo: in `cuda_warp`, un valore di A viene invece caricato una volta da una lane e riutilizzato per aggiornare tutti i k risultati associati alla stessa riga.

La mappatura del kernel assegna thread consecutivi alle colonne consecutive di Y . Per un j fissato, tutte le lane che lavorano sulla stessa riga i di Y richiedono quindi lo stesso elemento A_{ij} . Le lane associate alla riga successiva richiedono invece $A_{i+1,j}$, che in memoria row-major si trova a distanza `lda` elementi da A_{ij} .

Per $k < 32$, cioè quando una riga di Y contiene meno elementi della dimensione di un warp, un singolo warp comprende lane associate a più righe di Y e coinvolge quindi più gruppi di accessi ad A distanziati tra loro.

Gli accessi a X hanno invece un comportamento differente. Per ogni gruppo di lane associato alla stessa riga di Y , thread consecutivi leggono gli elementi consecutivi

$$X_{j0}, X_{j1}, \dots, X_{j,k-1},$$

che sono contigui in memoria nel layout row-major. Se il warp comprende più righe di Y , questa stessa sequenza di accessi a X si ripete.

All'aumentare di k , un numero crescente di lane del warp lavora sulla stessa riga di Y , fino al caso $k = 32$, in cui un intero warp corrisponde esattamente a una riga. A ogni iterazione j , tutte le lane richiedono quindi lo stesso elemento A_{ij} e, contemporaneamente, leggono i 32 elementi consecutivi

$$X_{j0}, \dots, X_{j31}.$$

Il parametro k influenza quindi direttamente il pattern di accesso del kernel. Questo aspetto sarà utile nel confronto con `cuda_warp`, dove la mappatura viene rovesciata: un intero warp viene assegnato a una sola riga di Y e le lane vengono distribuite lungo la dimensione j .

5.5 `cuda_warp`: un warp per riga

Il secondo backend modifica radicalmente l'approccio alla parallelizzazione. In `cuda_naive` ogni thread era responsabile di un singolo elemento Y_{ic} , ma in `cuda_warp`, invece, un intero warp collabora ed è responsabile del calcolo di una riga di Y .

Fissata la riga i , le 32 lane del warp si dividono la dimensione interna j . La lane l visita gli indici:

$$j = l, l + 32, l + 64, \dots$$

fino a raggiungere n_{loc} .

A ogni iterazione, lane consecutive accedono quindi a elementi consecutivi della stessa riga della matrice A :

$$A_{i,j}, A_{i,j+1}, \dots, A_{i,j+31}.$$

La mappatura produce così un pattern di accesso contiguo, favorevole alla coalescenza delle letture dalla memoria globale. A differenza di quanto accadeva in `cuda_naive`, questo comportamento non dipende da k , perché è la dimensione j , e non quella delle colonne del multivettore, a essere distribuita fra le lane.

La nuova mappatura consente soprattutto di sfruttare esplicitamente il riuso degli elementi di A . Una volta caricato un elemento:

$$a = A_{ij},$$

la lane non lo utilizza per una sola colonna del risultato, ma per aggiornare tutti i k accumulatori associati alla riga di Y :

$$\begin{aligned} acc[0] &+= aX_{j0}, \\ acc[1] &+= aX_{j1}, \\ &\vdots \\ acc[k-1] &+= aX_{j,k-1}. \end{aligned}$$

Un singolo caricamento di A_{ij} viene quindi riutilizzato nelle k moltiplicazioni che coinvolgono lo stesso elemento della matrice, ed è questa la principale differenza rispetto al primo approccio.

Al termine del ciclo su j , ciascuna lane possiede però soltanto una parte del risultato. Per ogni colonna c , la lane l ha accumulato i termini corrispondenti agli indici:

$$j = l, l + 32, l + 64, \dots$$

e il valore completo può essere scritto come

$$Y_{ic} = \sum_{l=0}^{31} acc_l[c]. \quad (5.6)$$

I 32 contributi vengono quindi ridotti all'interno del warp mediante `__shfl_down_sync`. Al termine, la lane 0 possiede i k valori completi della riga e li scrive in Y .

5.5.1 Gestione di k

I valori di k richiesti dalla traccia, 3, 6, 8, 20 e 32, sono implementati mediante cinque istanze template distinte del kernel. In queste versioni k è quindi noto a compile time: la dimensione dell'array degli accumulatori è costante e i cicli sulle colonne possono essere srotolati dal compilatore.

La stessa interfaccia supporta comunque valori arbitrari di k . Quando k non appartiene all'insieme precedente viene utilizzato un kernel runtime che elabora quattro colonne alla volta (`RUNTIME_TILE = 4`). In questo modo il numero di accumulatori simultaneamente mantenuti da ciascuna lane rimane limitato, pur continuando a riutilizzare ogni elemento di A per più colonne.

5.5.2 La maschera dello shuffle

Un dettaglio di correttezza riguarda la maschera utilizzata nelle operazioni di shuffle. Il kernel usa la maschera completa `0xffffffff`, perché ogni warp valido porta tutte le 32 lane fino alla riduzione. Il controllo sul limite delle righe viene infatti eseguito uniformemente sull'intero warp: se la riga assegnata non esiste, tutte le lane terminano prima; altrimenti tutte raggiungono la `__shfl_down_sync`.

Questo rimane vero anche quando $n_{\text{loc}} < 32$: le lane il cui indice non corrisponde ad alcun j mantengono semplicemente accumulatori nulli, ma partecipano comunque alla riduzione.

5.5.3 Il problema residuo: gli accessi a X

La distribuzione di j fra le lane rende favorevoli gli accessi ad A , ma determina contemporaneamente un pattern meno favorevole per X quando il multivettore è memorizzato in row-major.

Consideriamo un'iterazione nella quale le 32 lane del warp stanno elaborando gli indici consecutivi

$$j_0, j_0 + 1, \dots, j_0 + 31.$$

Durante l'aggiornamento di una stessa colonna c , le lane leggono quindi

$$X_{j_0,c}, X_{j_0+1,c}, \dots, X_{j_0+31,c}.$$

Si tratta di elementi appartenenti alla stessa colonna di X , ma a 32 righe differenti. Poiché X è memorizzata in row-major, due elementi consecutivi di questa sequenza non sono adiacenti in memoria: i loro indirizzi distano `ldx` elementi, cioè una quantità di byte pari a:

$$\text{ldx} \cdot \text{sizeof}(\text{scalar_t})$$

dove ricordiamo che nel caso compatto, utilizzato dal progetto, si ha sempre $\text{ldx} = k$.

Il comportamento è quindi opposto a quello delle letture di A : per la matrice A , lane consecutive leggono elementi consecutivi della stessa riga, mentre per X durante il calcolo di una colonna c , lane consecutive leggono elementi della stessa colonna ma appartenenti a righe differenti.

All'aumentare di k aumenta anche la distanza in memoria fra gli elementi di X richiesti da lane consecutive. Le richieste del warp risultano quindi progressivamente più sparse e possono richiedere un numero maggiore di transazioni di memoria rispetto a un accesso contiguo.

Il kernel ha quindi risolto il principale limite di `cuda_naive`, cioè il mancato riuso esplicito di A , ma ha lasciato un nuovo collo di bottiglia nel pattern di accesso a X . Le due varianti introdotte nelle sezioni successive affrontano esattamente questo problema senza modificare la mappatura warp-per-row.

5.6 Primo rimedio: cambio di layout di X

Il problema descritto nella sezione precedente nasce dall'incompatibilità fra la mappatura delle lane e il layout row-major di X . Le lane di `cuda_warp` avanzano lungo la dimensione j , mentre nel layout row-major sono gli elementi che differiscono per l'indice di colonna c a essere contigui in memoria.

Il primo rimedio consiste quindi nel modificare la rappresentazione di X senza cambiare la mappatura del kernel. Durante il preprocessing, il blocco locale del multivettore viene riscritto per colonne, in modo che

$$X_{jc} = \mathbf{x_loc}[c \cdot n_{\text{loc}} + j]. \quad (5.7)$$

In questa rappresentazione, tutti gli elementi appartenenti a una stessa colonna di X sono consecutivi in memoria. Di conseguenza, quando le 32 lane del warp elaborano gli indici

$$j_0, j_0 + 1, \dots, j_0 + 31$$

per una stessa colonna c , leggono anche 32 posizioni consecutive del buffer:

$$X_{j_0,c}, X_{j_0+1,c}, \dots, X_{j_0+31,c}.$$

Lo stride fra gli indirizzi richiesti da lane consecutive passa quindi da k elementi (nel layout row-major) a un singolo elemento (in questo layout). La mappatura warp-per-row e il riuso di A introdotti da `cuda_warp` rimangono invariati: cambia esclusivamente il modo in cui X è organizzata e indirizzata in memoria.

5.7 `cuda_warp_smem`: staging di X in shared memory

Il secondo rimedio conserva sia la mappatura warp-per-row di `cuda_warp` sia il normale layout row-major di X . La differenza riguarda il percorso seguito dagli elementi del multivettore: invece di essere letti direttamente dalla global memory durante il calcolo, vengono prima caricati cooperativamente in **shared memory** e successivamente consumati dai warp del blocco.

Il percorso di un elemento di X diventa quindi:

global memory $\longrightarrow$ shared memory $\longrightarrow$ registri,

mentre in `cuda_warp` il passaggio intermedio non era presente.

La struttura del calcolo rimane per il resto invariata. Ogni warp è ancora responsabile di una riga di Y , le sue 32 lane si dividono gli indici j , ogni elemento di A viene riutilizzato per aggiornare i k accumulatori e la riduzione finale avviene mediante `__shfl_down_sync`.

5.7.1 Caricamento cooperativo di X

Il caricamento del tile di X non utilizza la stessa mappatura con cui viene successivamente eseguito il prodotto. In questa prima fase partecipano tutti i thread del blocco e gli elementi del tile vengono assegnati mediante un indice lineare.

Indicando con q tale indice, vengono ricavati

$$r = \left\lfloor \frac{q}{k} \right\rfloor, \quad c = q \bmod k,$$

e il thread carica l'elemento X_{rc} nella corrispondente posizione del tile in shared memory.

Con X compatta in row-major, thread consecutivi ricevono valori consecutivi di q e leggono quindi posizioni consecutive del buffer. Il caricamento dalla global memory segue così l'ordine naturale con cui X è memorizzata, evitando il pattern strided che caratterizzava `cuda_warp`.

Una volta completato il caricamento cooperativo, una `__syncthreads()` garantisce che il tile sia interamente disponibile prima che i warp inizino a utilizzarlo. La shared memory è infatti una risorsa comune a tutti i thread dello stesso blocco.

A questo punto il calcolo riprende la stessa struttura di `cuda_warp`. Per ogni warp, la lane l elabora gli indici

$$j = l, l + 32, l + 64, \dots$$

ma gli elementi di X corrispondenti vengono ora letti dal tile in shared memory anziché direttamente dalla global memory.

5.7.2 Riutilizzo del tile fra i warp del blocco

L'introduzione della shared memory produce inoltre un secondo effetto. I warp appartenenti allo stesso blocco calcolano righe differenti di Y , e utilizzano quindi righe differenti di A , ma tutti devono attraversare la stessa matrice X .

Nel backend `cuda_warp`, ciascun warp emette le proprie letture di X . In `cuda_warp_smem`, invece, un tile viene caricato una sola volta dall'intero blocco e viene poi utilizzato da tutti i suoi warp.

Se W indica il numero di warp per blocco, il numero di elementi di X caricati esplicitamente dalla global memory passa quindi, idealmente, da

$$m_{\text{loc}} n_{\text{loc}} k$$

a

$$\left\lceil \frac{m_{\text{loc}}}{W} \right\rceil n_{\text{loc}} k.$$

Per esempio, con il valore predefinito `BLOCK_THREADS=256`, il blocco contiene $W = 8$ warp.

Questa riduzione descrive il numero di letture esplicitamente richieste dal kernel alla gerarchia di memoria globale. Non implica però che il traffico effettivo verso la DRAM si riduca dello stesso fattore, perché una parte delle letture di `cuda_warp` può già essere servita dalle cache della GPU. Il vantaggio effettivo della shared memory deve quindi essere determinato sperimentalmente.

5.7.3 Bank conflict e padding della shared memory

Una volta caricato il tile, il problema non è più la coalescenza degli accessi alla global memory, ma il modo in cui gli indirizzi richiesti dalle lane vengono distribuiti sui banchi della shared memory.

La shared memory è organizzata in 32 banchi, e parole consecutive da 32 bit vengono assegnate a banchi consecutivi. Se più lane richiedono indirizzi differenti appartenenti allo stesso banco, gli accessi non possono essere serviti simultaneamente e vengono parzialmente serializzati.

Durante il caricamento cooperativo del tile, thread consecutivi scrivono elementi consecutivi della shared memory. Questo pattern segue linearmente il buffer e non introduce il problema che interessa il successivo consumo del tile.

Durante il calcolo, invece, la lane l accede alla posizione

$$X_{\text{tile}}[l \cdot \text{tile_row_stride} + c],$$

dove

$$\text{tile_row_stride} = k + \text{SMEM_PAD}.$$

Lane consecutive leggono quindi elementi separati da un'intera riga del tile. Se questo stride produce una mappatura periodica sugli stessi banchi, si genera un bank conflict.

Il caso $k = 32$ in doppia precisione è particolarmente sfavorevole senza padding. Con

$$\text{tile_row_stride} = 32,$$

la distanza fra gli elementi letti da lane consecutive corrisponde a 64 parole da 32 bit e quindi

$$64 \equiv 0 \pmod{32}.$$

Gli accessi ricadono così ripetutamente sugli stessi banchi.

Aggiungendo un elemento di padding per riga si ottiene invece

$$\text{tile_row_stride} = 33,$$

e la distanza diventa

$$66 \equiv 2 \pmod{32},$$

distribuendo gli accessi sui diversi banchi e rimuovendo il caso patologico precedente.

Il padding non rappresenta tuttavia una scelta universalmente ottimale per ogni k . Il suo effetto dipende dallo stride risultante. Per questo `SMEM_PAD` è un parametro di compilazione e il progetto mantiene separate le configurazioni con e senza padding, così da misurarne direttamente l'effetto.

5.7.4 Scelta della dimensione del tile

Il blocco locale di X non viene trasferito interamente in shared memory. La dimensione della shared memory disponibile per blocco è limitata e contribuisce, insieme ai registri e al numero di thread, a determinare quanti blocchi possono risiedere contemporaneamente su uno SM.

Il backend divide quindi X in tile contenenti `x_rows_per_tile` righe. Indicando questa quantità con R , la memoria richiesta da un tile è

$$R(k + \text{SMEM_PAD}) \text{sizeof}(\text{scalar_t}).$$

Il valore di R modifica la quantità di shared memory occupata da ogni blocco, ma non il volume complessivo di X che quel blocco deve attraversare. Un tile più piccolo produce semplicemente un numero maggiore di iterazioni del ciclo di tiling.

La dimensione del tile influenza invece direttamente la frequenza delle sincronizzazioni. Ogni tile richiede due chiamate a `__syncthreads()`: la prima impedisce che il tile precedente venga sovrascritto mentre qualche warp lo sta ancora utilizzando, la seconda garantisce che il nuovo tile sia stato completamente caricato prima di iniziare il calcolo. Aumentando il numero di righe per tile diminuisce quindi il numero di barriere incontrate durante l'attraversamento di X .

Non è tuttavia possibile aumentare arbitrariamente il tile. Una maggiore quantità di shared memory per blocco può ridurre il numero di blocchi residenti contemporaneamente su uno SM e quindi l'occupancy. La scelta implementata nel backend cerca perciò un compromesso: utilizzare un tile sufficientemente grande senza fare della shared memory un ulteriore vincolo alla residenza dei blocchi.

La funzione `plan_tile` esegue questa scelta una sola volta durante `local_gemm_create`, quindi fuori dalla regione cronometrata.

Come primo passo viene interrogata `cudaOccupancyMaxActiveBlocksPerMultiprocessor` assumendo nulla la shared memory dinamica. Il risultato rappresenta il numero di blocchi per SM consentito dal kernel compilato in assenza del vincolo aggiuntivo introdotto dal tile.

La shared memory disponibile per SM viene quindi suddivisa rispetto a questo numero di blocchi, ottenendo un budget iniziale per ciascun blocco. Da tale budget viene ricavato il numero di righe che possono essere contenute nel tile.

Il valore così ottenuto viene infine verificato interrogando nuovamente l'API di occupancy con la quantità effettiva di shared memory richiesta. Se il numero di blocchi residenti risulta inferiore all'obiettivo iniziale, la dimensione del tile viene progressivamente ridotta.

La pianificazione viene eseguita sull'istanza concreta del kernel specializzato per il valore di k corrente. Questa distinzione è necessaria perché le diverse istanze possono avere richieste differenti di registri e quindi differenti limiti di occupancy.

Il parametro `TILE_GRANULARITY` controlla infine l'arrotondamento del numero di righe scelto dal piano. Il valore predefinito è 32, pari alla dimensione del warp.

Un tile con un numero di righe multiplo di 32 permette a tutte le lane di partecipare a ciascun passo completo del ciclo su j . Questo non è però un requisito di correttezza: il kernel gestisce anche tile con un numero di righe non multiplo di 32.

L'arrotondamento introduce quindi un compromesso. Una granularità elevata può lasciare inutilizzata una parte del budget di shared memory e aumentare il numero di tile, mentre una granularità più fine può produrre un ultimo passo nel quale alcune lane non hanno elementi da elaborare. Il parametro viene mantenuto configurabile proprio per poter misurare questo compromesso.

5.7.5 Sincronizzazione e warp inattivi

L'uso di `__syncthreads()` introduce un vincolo di correttezza che non era presente in `cuda_warp`. La barriera coinvolge tutti i thread del blocco e può essere superata soltanto quando tutti i thread che partecipano al blocco l'hanno raggiunta.

L'ultimo blocco della griglia può contenere più warp di quanti siano necessari per coprire le ultime righe di Y . In `cuda_warp` questi warp possono terminare immediatamente quando la riga assegnata supera m_{loc} .

In `cuda_warp_smem` questo non è possibile prima del completamento del ciclo sui tile. Se alcuni warp uscissero mentre gli altri continuano a eseguire le `__syncthreads()`, i thread rimanenti potrebbero attendere indefinitamente a una barriera che gli altri non raggiungeranno mai.

Per questo il kernel associa a ciascun warp un flag `active`. Anche i warp ai quali non corrisponde una riga valida partecipano al caricamento cooperativo e attraversano tutte le sincronizzazioni del blocco. Soltanto l'accumulo dei prodotti e la successiva scrittura di Y vengono eseguiti dai warp attivi.

Terminato l'ultimo tile, quando non rimangono più barriere di blocco, i warp inattivi possono infine uscire prima della riduzione finale.

5.8 Due rimedi allo stesso problema

Le soluzioni descritte nelle sezioni 5.6 e 5.7 nascono dallo stesso problema: in `cuda_warp`, con X row-major, lane consecutive leggono elementi della stessa colonna di X separati da uno stride pari a k elementi.

I due approcci intervengono però in punti differenti.

La variante `X_LAYOUT=column` modifica la rappresentazione del dato prima dell'esecuzione del kernel. Disponendo consecutivamente gli elementi di una stessa colonna di X , la mappatura warp-per-row può essere mantenuta inalterata e le lane trovano direttamente in global memory gli elementi richiesti in posizioni consecutive. Il costo associato a questa soluzione è la conversione preliminare del multivettore.

`cuda_warp_smem`, invece, conserva X nel normale layout row-major e modifica il percorso attraverso il quale i dati raggiungono le lane. Gli elementi vengono prima caricati cooperativamente e in modo contiguo dalla global memory alla shared memory, quindi letti dal tile durante il calcolo. Questa soluzione evita la conversione del dato, ma introduce il costo dello staging, delle sincronizzazioni e dell'utilizzo della shared memory.

Le due implementazioni rappresentano quindi due strategie alternative studiate separatamente nel progetto: la prima modifica il layout del dato, la seconda modifica il meccanismo di accesso mantenendone invariata la rappresentazione.

La combinazione dei due approcci non è stata inclusa tra i backend sperimentali, poiché l'obiettivo del confronto è isolare separatamente l'effetto delle due strategie rispetto alla stessa baseline `cuda_warp`.

5.9 cublas: il riferimento esterno

L'ultimo backend GPU non introduce un nuovo kernel sviluppato nel progetto, ma utilizza la routine GEMM della libreria cuBLAS. Il suo ruolo è fornire un riferimento esterno con cui confrontare le implementazioni CUDA precedenti.

Perché il confronto sia significativo, cuBLAS viene inserita dietro la stessa interfaccia degli altri backend. La creazione del contesto, le allocazioni device e il trasferimento iniziale di A avvengono quindi in `local_gemm_create`, mentre ogni invocazione di `local_gemm` esegue, nello stesso ordine degli altri backend GPU, il trasferimento H2D di X , il prodotto sul device e il trasferimento D2H di Y .

Anche la misura segue gli stessi confini: il tempo del solo prodotto cuBLAS viene delimitato mediante `cudaEvent`, mentre i trasferimenti vengono misurati separatamente. Il confronto con i kernel sviluppati nel progetto avviene quindi nelle stesse condizioni sperimentali.

5.9.1 Adattamento fra row-major e column-major

Il progetto memorizza le matrici in row-major, mentre l'interfaccia GEMM tradizionale di cuBLAS interpreta i propri operandi in column-major. Non è tuttavia necessario convertire o trasporre fisicamente i dati.

Cioè gli stessi dati che rappresentano una matrice in row-major, se interpretati come column-major senza modificarne il contenuto in memoria, rappresentano la sua trasposta. Cambia soltanto la convenzione con cui gli indici vengono associati agli indirizzi.

Di conseguenza, i buffer del progetto possono essere reinterpretati da cuBLAS come

$$A_{\text{cm}} = A_{\text{rm}}^T, \quad X_{\text{cm}} = X_{\text{rm}}^T, \quad Y_{\text{cm}} = Y_{\text{rm}}^T.$$

Applicando questa reinterpretazione al prodotto desiderato si ottiene:

$$Y_{\text{cm}} = Y_{\text{rm}}^T = (A_{\text{rm}} X_{\text{rm}})^T = X_{\text{rm}}^T A_{\text{rm}}^T = X_{\text{cm}} A_{\text{cm}}.$$

Quindi la chiamata cuBLAS deve utilizzare X come primo operando e A come secondo. Nella convenzione utilizzata internamente da cuBLAS si ha:

$$C_{M \times N} = A_{M \times K} B_{K \times N},$$

con:

$$C_{M \times N} = Y_{\text{cm}}^T, \quad A_{M \times K} = X_{\text{cm}}^T, \quad B_{K \times N} = A_{\text{cm}}^T$$

e le dimensioni passate alla libreria sono:

$$M = k, \quad N = m_{\text{loc}}, \quad K = n_{\text{loc}}.$$

Entrambi gli operandi vengono utilizzati con `CUBLAS_OP_N`: la trasposizione non viene eseguita dalla libreria, ma è già implicita nella reinterpretazione column-major degli stessi buffer.

5.9.2 Significato del confronto

Il backend cuBLAS costituisce un riferimento esterno e non un ulteriore passo della sequenza di ottimizzazione sviluppata nel progetto. La libreria sceglie internamente l'implementazione del prodotto e il progetto non controlla direttamente parametri quali tiling, distribuzione del lavoro od occupancy.

Il confronto viene quindi utilizzato in senso sperimentale: permette di stabilire quale prestazione ottengano i kernel specializzati del progetto rispetto a una routine GEMM di libreria eseguita sulla stessa GPU e con gli stessi dati.

Nel problema considerato una delle dimensioni del prodotto, k , è intenzionalmente piccola rispetto a m e n . Questo rende il confronto particolarmente interessante, perché i kernel sviluppati nel progetto sono costruiti esplicitamente attorno alla struttura di un multivettore stretto, mentre cuBLAS riceve il problema attraverso la propria interfaccia GEMM generale. L'eventuale vantaggio di una delle due soluzioni deve però essere stabilito dalle misure, non assunto a priori.

5.10 Più rank MPI sulla stessa GPU

La piattaforma utilizzata per le misure dispone di una sola GPU e i backend CUDA del progetto selezionano esplicitamente il device 0. Quando l'applicazione viene eseguita con più rank MPI sullo stesso nodo, ogni processo crea quindi il proprio contesto CUDA sulla medesima scheda.

In assenza di NVIDIA Multi-Process Service (MPS), il lavoro proveniente da contesti CUDA appartenenti a processi differenti non viene eseguito contemporaneamente sugli SM, ma viene alternato mediante time slicing. I rank possono quindi procedere contemporaneamente sul lato CPU, ma competono per l'esecuzione sulla stessa GPU.

Questa situazione modifica il significato delle misure ottenute mediante `cudaEvent`. Per ciascun rank, t_{kernel} misura l'intervallo compreso fra gli eventi che delimitano il proprio kernel. Se il contesto viene sospeso durante tale intervallo per consentire l'esecuzione di un altro rank, il tempo osservato risente anche della contesa per la GPU.

Di conseguenza, con più rank per GPU, t_{kernel} non deve essere interpretato come il tempo che lo stesso kernel otterrebbe disponendo della scheda in modo esclusivo.

Il driver conserva due prospettive differenti sulla fase locale.

La prima è il tempo del kernel restituito dal backend CUDA. Al termine delle ripetizioni, come per le altre fasi, viene considerato il massimo fra i rank, poiché l'operazione distribuita non può essere considerata conclusa prima del processo più lento. Questa misura rimane utile per osservare il comportamento dei singoli backend, ma con più contesti sulla stessa GPU è influenzata dalla loro contesa.

Per misurare invece il tempo effettivamente richiesto dall'intero gruppo di rank per completare la fase locale, il benchmark mette a disposizione la modalità `-group-timing`. In questo caso

una barriera MPI viene posta immediatamente prima di `local_gemm` e una seconda subito dopo:

$$\text{barriera} \longrightarrow \text{fase locale dei rank} \longrightarrow \text{barriera}.$$

L'intervallo fra le due barriere misura quindi il tempo trascorso dal momento in cui tutti i rank sono pronti fino al completamento dell'ultimo. È questo il denominatore utilizzato per la metrica `gflops_local_group`, che descrive il throughput complessivo del gruppo durante la fase locale.

Per le campagne dedicate alla caratterizzazione del kernel viene quindi utilizzato un solo rank MPI. In questo modo il backend dispone esclusivamente della GPU e differenze dovute, per esempio, alla dimensione del blocco CUDA, al padding della shared memory o alla scelta del layout di X non vengono confuse con la schedulazione fra contesti differenti.

A parità di dimensioni globali M , N e k , il numero complessivo di operazioni rimane lo stesso, ma con un solo rank cambia naturalmente la dimensione del blocco locale assegnato al processo. Le misure a rank singolo devono quindi essere interpretate come caratterizzazione del backend sulla GPU dedicata, mentre quelle multi-rank descrivono l'esecuzione della configurazione MPI effettiva sulla piattaforma disponibile.